

Lập trình trò chơi

GS.TS.Nguyễn Thanh Thủy



Trò chơi với TTNT

- **Nghiên cứu trò chơi là một lĩnh vực được quan tâm trong trí tuệ nhân tạo:**
 - Bài toán đặt ra với trò chơi là không tầm thường
 - Người chơi phải có trí tuệ
 - Có những trò chơi phức tạp như cờ vua, cờ tướng,...
 - Yêu cầu phải đưa ra quyết định trong thời gian hạn chế
 - Các trò chơi thường:
 - Phát biểu chính
 - Không gian hạn chế và có thể khám phá
 - Nghiên cứu trò chơi cũng là một phương tiện cho phép so sánh trí tuệ máy và trí tuệ con người

Trò chơi với TTNT

	Deterministic	Chance
admissible, perfect info	Checkers, Chess, Go, Othello	Backgammon, Monopoly
not admissible, imperfect info	what kinds of games here?	Bridge, Poker, Scrabble

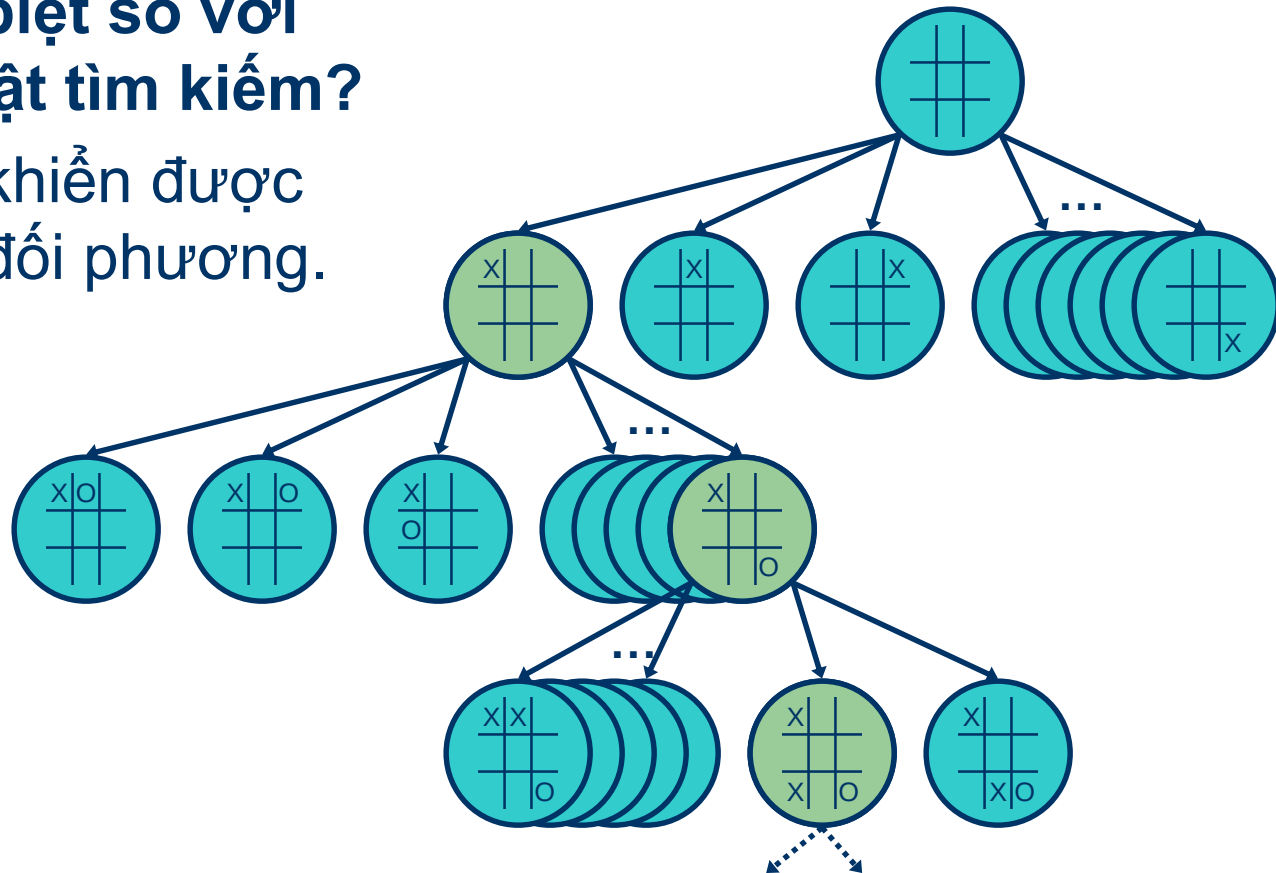
Lý thuyết trò chơi và bài toán tìm kiếm

- **Bài toán chơi cờ:**
 - cờ tướng, cờ quốc tế, cờ vây,...
- **Biểu diễn không gian trạng thái:**
 - **Trạng thái:** Một cách sắp xếp các quân cờ
 - **Chuyển trạng thái:** Một nước đi hợp lệ
 - **Trạng thái xuất phát**
 - **Trạng thái kết thúc**

Cây trò chơi

**Điểm khác biệt so với
các giải thuật tìm kiếm?**

Không điều khiển được
cách đi của đối phương.



Độ phức tạp của giải thuật trò chơi

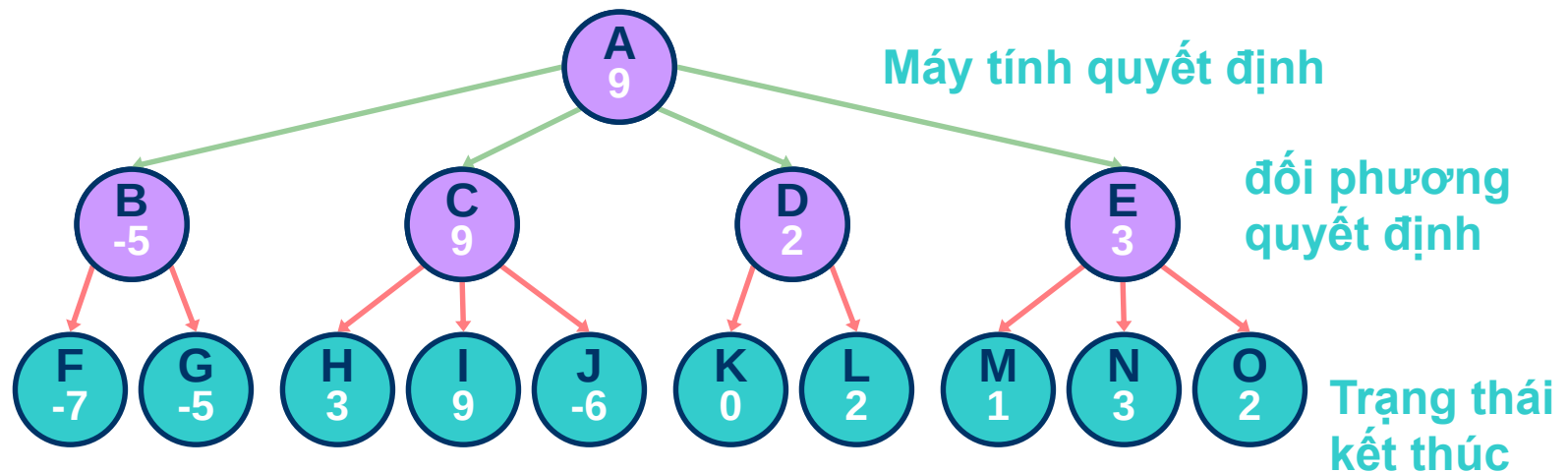
- Giải thiết có thể dự báo được nước đi của đối phương
- Độ phức tạp
 - Trường hợp xấu nhất: $O(b^d)$
 - b : độ phân nhánh
 - d : độ sâu
 - Tic-Tac-Toe:
 - $5^9 = 1,953,125$ states
 - Cờ vua:
 - $b^d \sim 35^{100} \sim 10^{154}$ states, “only” $\sim 10^{40}$ legal states
- * *Các trò chơi thường có không gian trạng thái cực kỳ lớn*

Giải thuật tham lam

- * *Hàm giá : Hàm ánh xạ giữa mỗi trạng thái của trò chơi với một giá trị thể hiện lợi thế của trạng thái đó*
 - dương : lợi thế
 - âm : bất lợi
 - 0 : hoà
 - Giá trị đặc biệt (cho thắng hoặc thua):
 - $-\infty$, $+\infty$
 - -1.0 , +1.0

Giải thuật tham lam

- Mở rộng cây tìm kiếm cho tới các nút kết thúc
- Đánh giá các nút kết thúc
- Chọn nước đi là nước sẽ dẫn đến giá tốt nhất



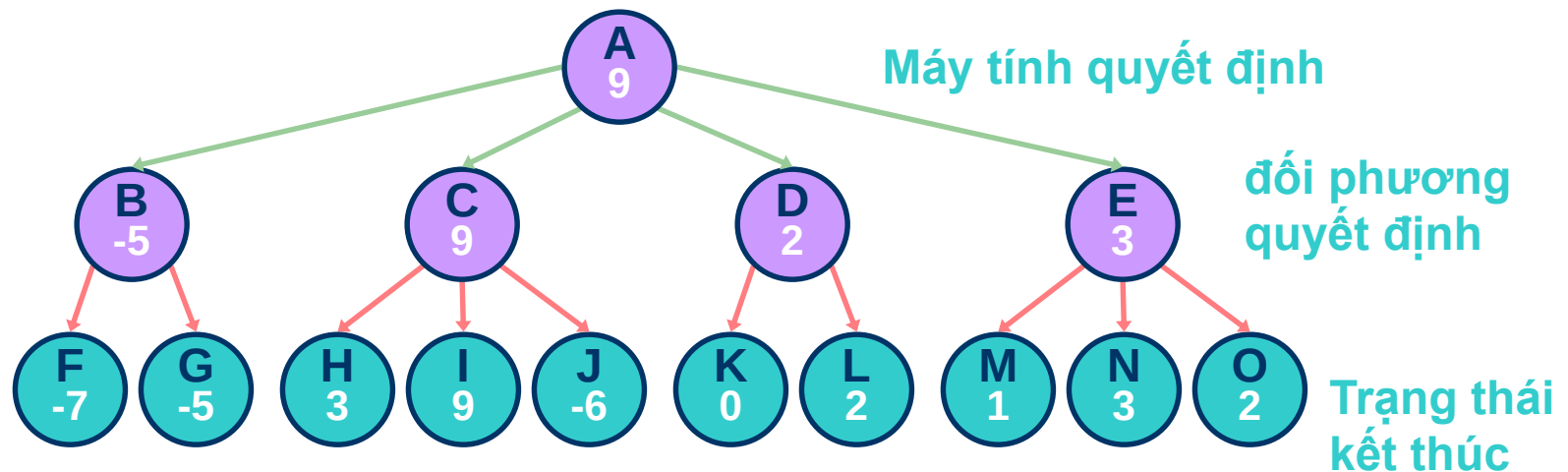
Tìm kiếm tham lam

- **Nhược điểm:**

- Bỏ qua sự quyết định của đối phương

Máy tính chọn C

Đối phương chọn J

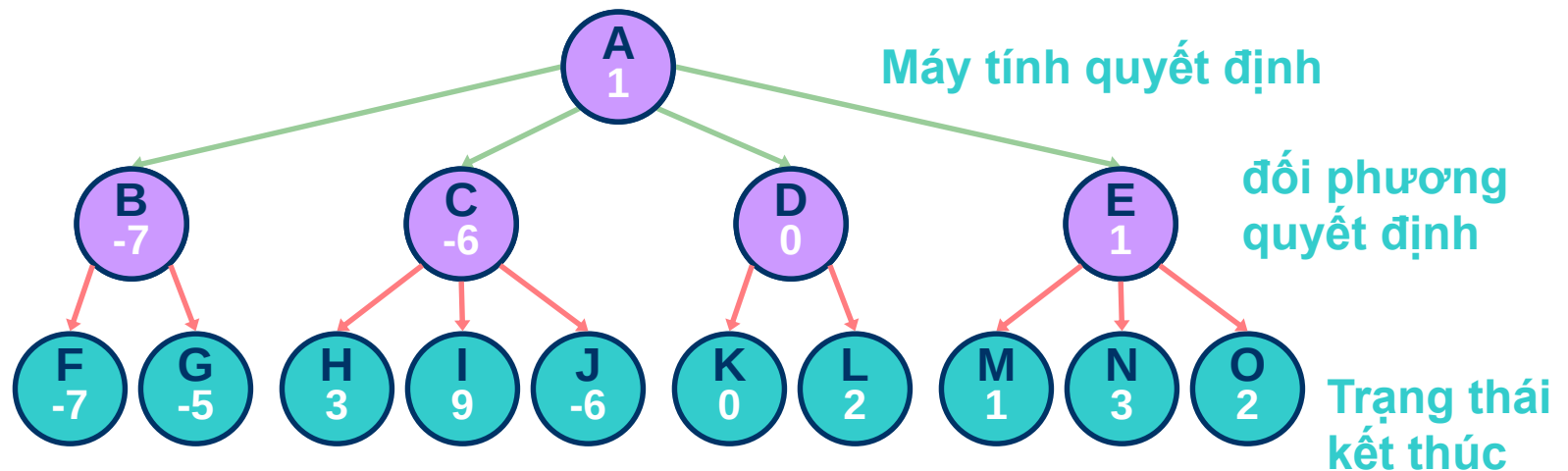


Giải thuật MINIMAX

- Chiến lược đặt ra: Giả sử đối phương quyết định nước đi tốt nhất
 - Hai người chơi cho tới khi kết thúc
 - Máy tính sẽ chọn nước đi có lợi nhất (có giá lớn nhất)
 - Đối phương cũng chọn nước đi có lợi nhất (có giá nhỏ nhất)

Giải thuật MINIMAX

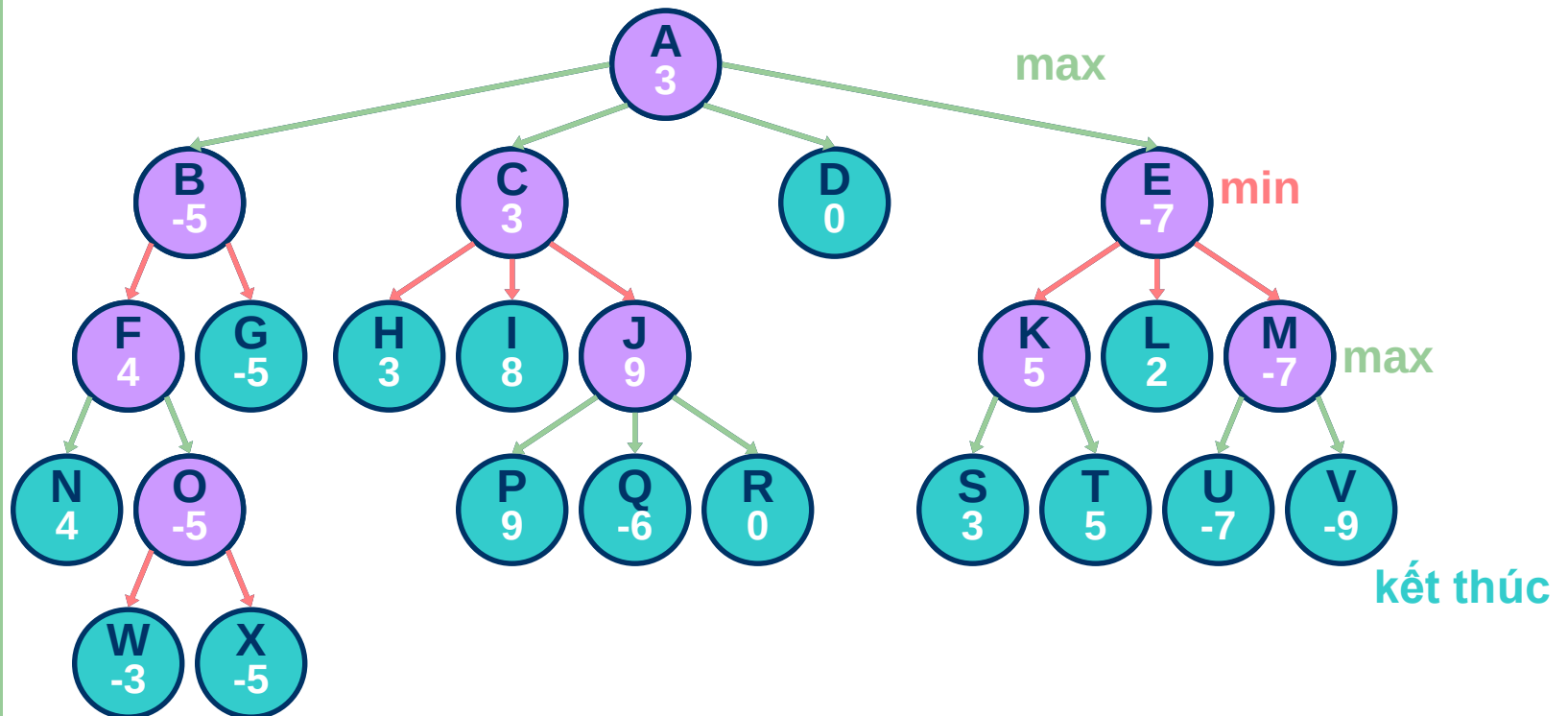
- Máy tính ra quyết định dựa trên dự báo về đối phương thông minh



Lan truyền giá trên cây trò chơi

- Mở rộng cây đến nút kết thúc
- Đánh giá các nút lá
- Lan truyền giá theo chiến thuật
 - Bắt đầu từ nút lá
 - Gán giá trị cho nút cha bằng giá trị của :
 - nút con nhỏ nhất tương ứng với nước đi của đối phương
 - nút con lớn nhất tương ứng với nước đi của máy tính

Ví dụ



Giải thuật MINIMAX tổng quát

Với mỗi bước đi của máy tính:

1. Tiến hành duyệt theo chiều sâu cho tới nút kế thúc
2. Đánh giá nút lá
3. Lan truyền giá trị
máy tính đi :lan truyền giá trị lớn nhất
đối phương đi: lan truyền giá trị nhỏ nhất
4. Lựa chọn nước đi tương ứng nút có giá trị lớn nhất

Độ phức tạp tính toán của giải thuật MINIMAX

Giả sử độ sâu là d , độ phân nhánh là b

- Không gian lưu trữ $O(bd)$
- Thời gian tìm kiếm $O(b^d)$

* *Thời gian tìm kiếm là vấn đề mấu chốt khi các trò chơi thường hạn chế thời gian đi*

Độ phức tạp tính toán của giải thuật MINIMAX

- Các giải thuật MINIMAX tổng quát thường không thực tế
 - Chỉ tìm kiếm tới độ sâu xác định nào đó
 - Các hàm giá thường không sử dụng cho các nút không kết thúc → hàm giá cho các nút không kết thúc
- * ***Static board evaluator (SBE)*** : Là hàm giá heuristic sử dụng cho các nút không kết thúc

Static Board Evaluator (SBE)

- * *SBE sử dụng để dự báo về một trạng thái*

- phản ánh khả năng thắng cuộc
- dễ dàng tính toán

- **For example, Chess:**

$$SBE = \alpha * materialBalance + \beta * centerControl + \gamma * ...$$

material balance = Value of white pieces - Value of black pieces

pawn = 1, rook = 5, queen = 9, ...

Hàm SBE

- Thường được đánh giá dựa trên hiệu số :
 - Lợi thế của máy tính
 - Lợi thế của đối phương
- Hàm SBE phải thống nhất với hàm giá của nút kết thúc

Minimax Algorithm với SBE

```
int minimax (Node s, int depth, int limit) {
    Vector v = new Vector();
    if (isTerminal(s) || depth == limit) // base case
        return(staticEvaluation(s));
    else {
        // do minimax on successors of s and save their values
        while (s.hasMoreSuccessors())
            v.addElement(minimax(s.getNextSuccessor(), depth+1, limit));
        if (isComputersTurn(s))
            return maxOf(v); // computer's move return max of children
        else
            return minOf(v); // opponent's move return min of children
    }
}
```

Đánh giá giải thuật MINIMAX

- **Đối với cờ vua:**
 - Với độ sâu bằng 8 dễ dàng bị đánh bại với người chơi trung bình
 - Độ sâu 16 tương đương với một người chơi có kinh nghiệm
- **Khó khăn**
 - Độ phức tạp giải thuật
 - Đánh giá SBE

Phép tỉa Alpha-Beta

- Một vài nhánh không có triển vọng khi đối đầu với người chơi thông minh
- Phép tỉa cho phép bỏ qua những nhánh này
- ý tưởng:
 - **alpha**
 - giá trị lớn nhất hiện tại
 - Biên dưới
 - **beta**
 - giá trị nhỏ nhất hiện tại
 - biên trên

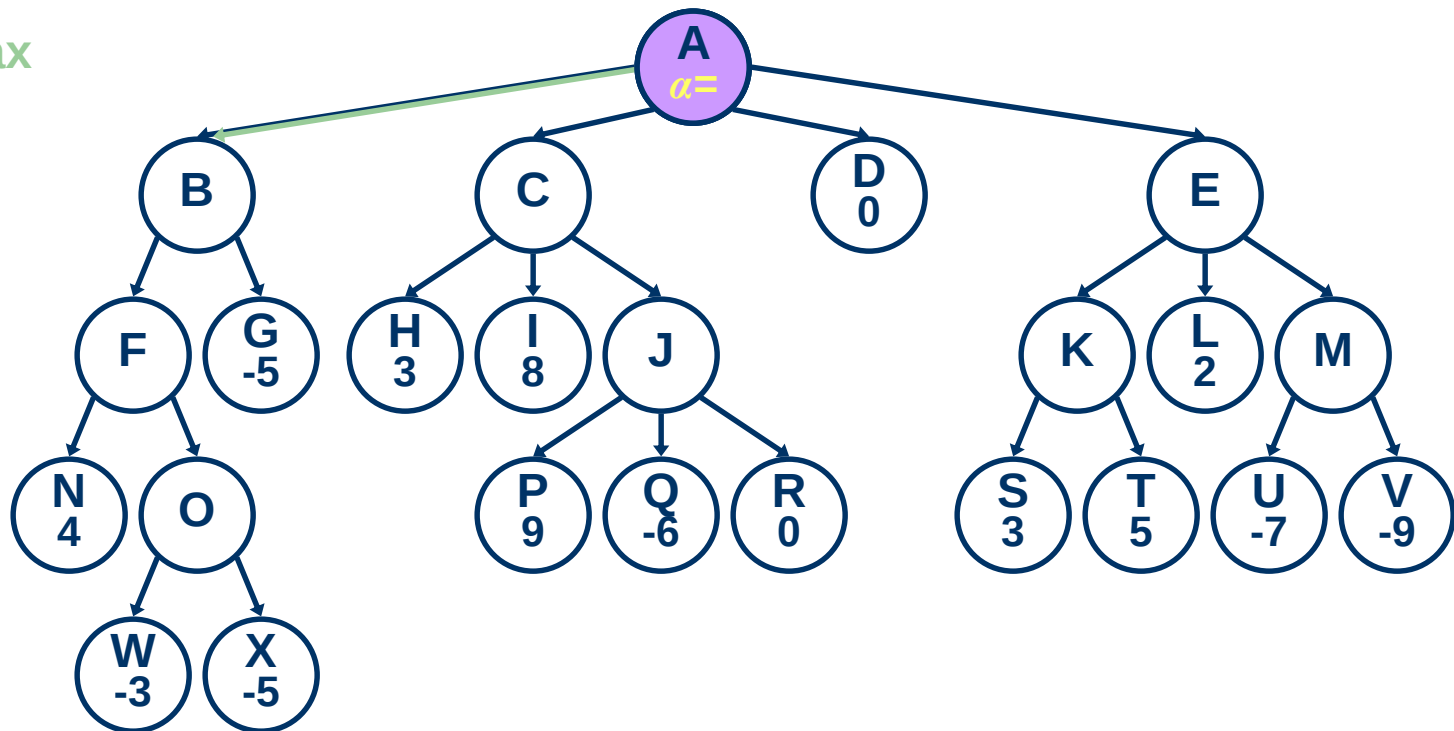
Phép tỉ Alpha-Beta

- **Phép tỉ xảy ra:**
 - khi $\alpha \geq \beta$ của nút cha
 - $\beta \leq \alpha$ của nút cha

Ví dụ

`minimax(A, 0, 4)`

max

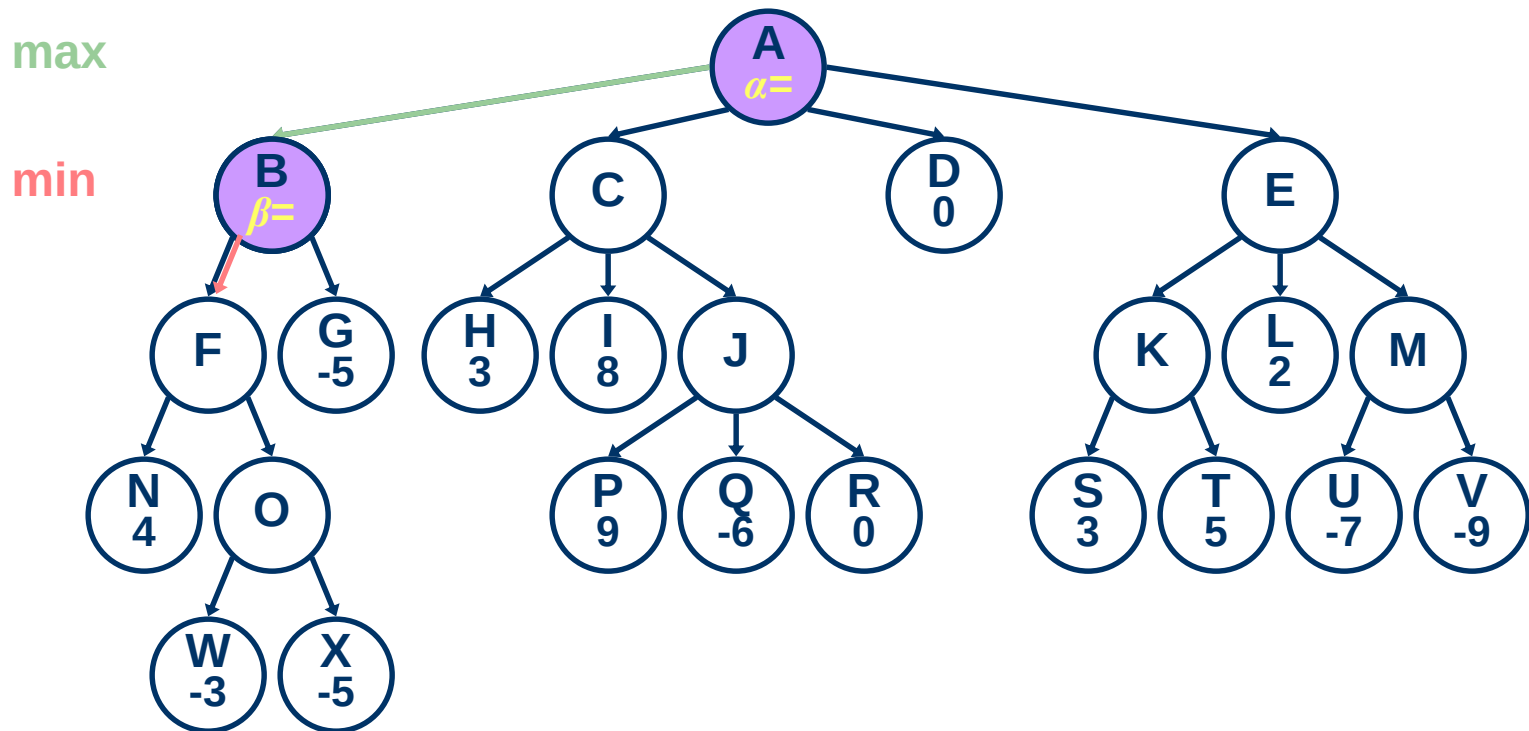


Call
Stack

A

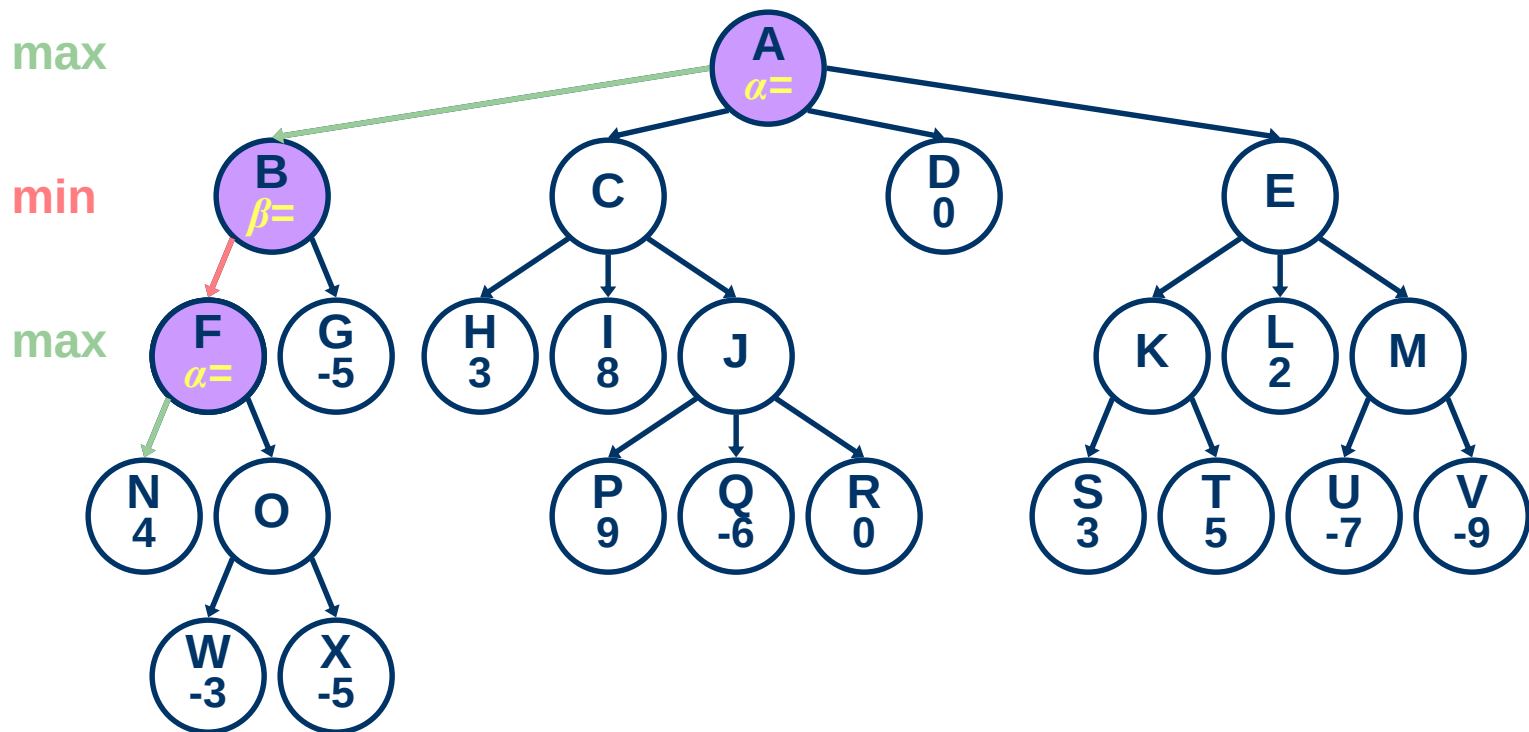
Ví dụ

`minimax (B, 1, 4)`



Ví dụ

`minimax(F, 2, 4)`



Call
Stack

F
B
A

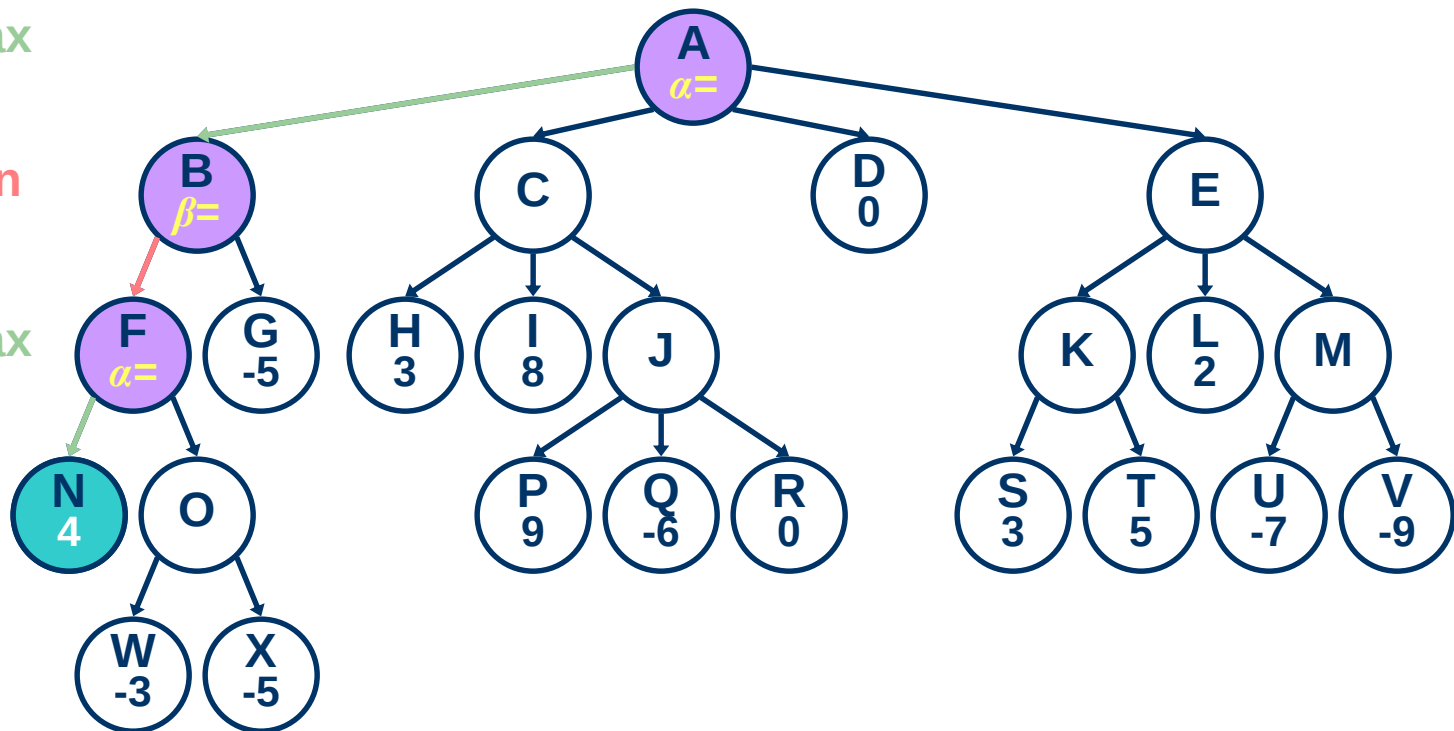
Ví dụ

$\text{minimax}(N, 3, 4)$

max

min

max



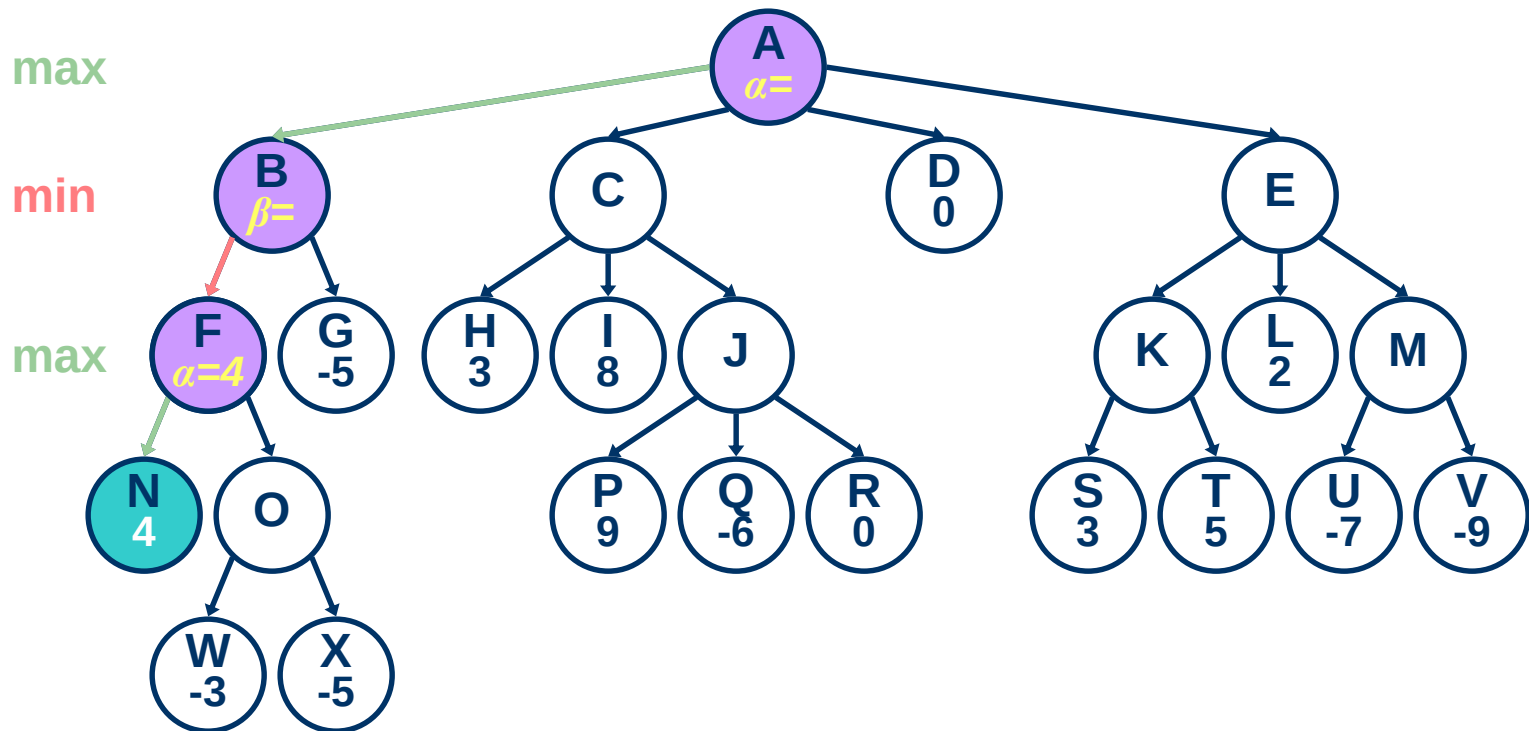
Call
Stack

N
F
B
A

Ví dụ

`minimax(F, 2, 4)`

`alpha = 4`

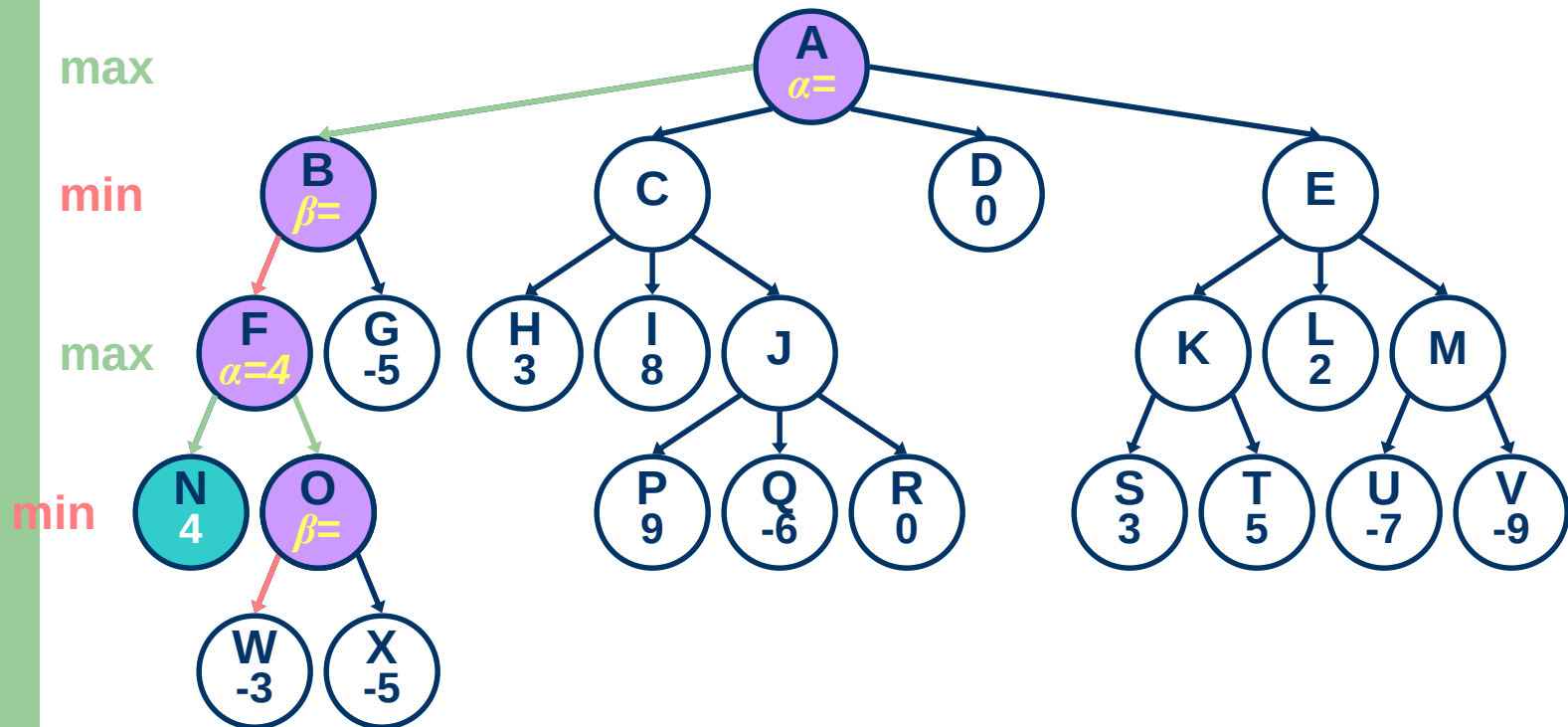


Call
Stack

F
B
A

Ví dụ

`minimax(0, 3, 4)`

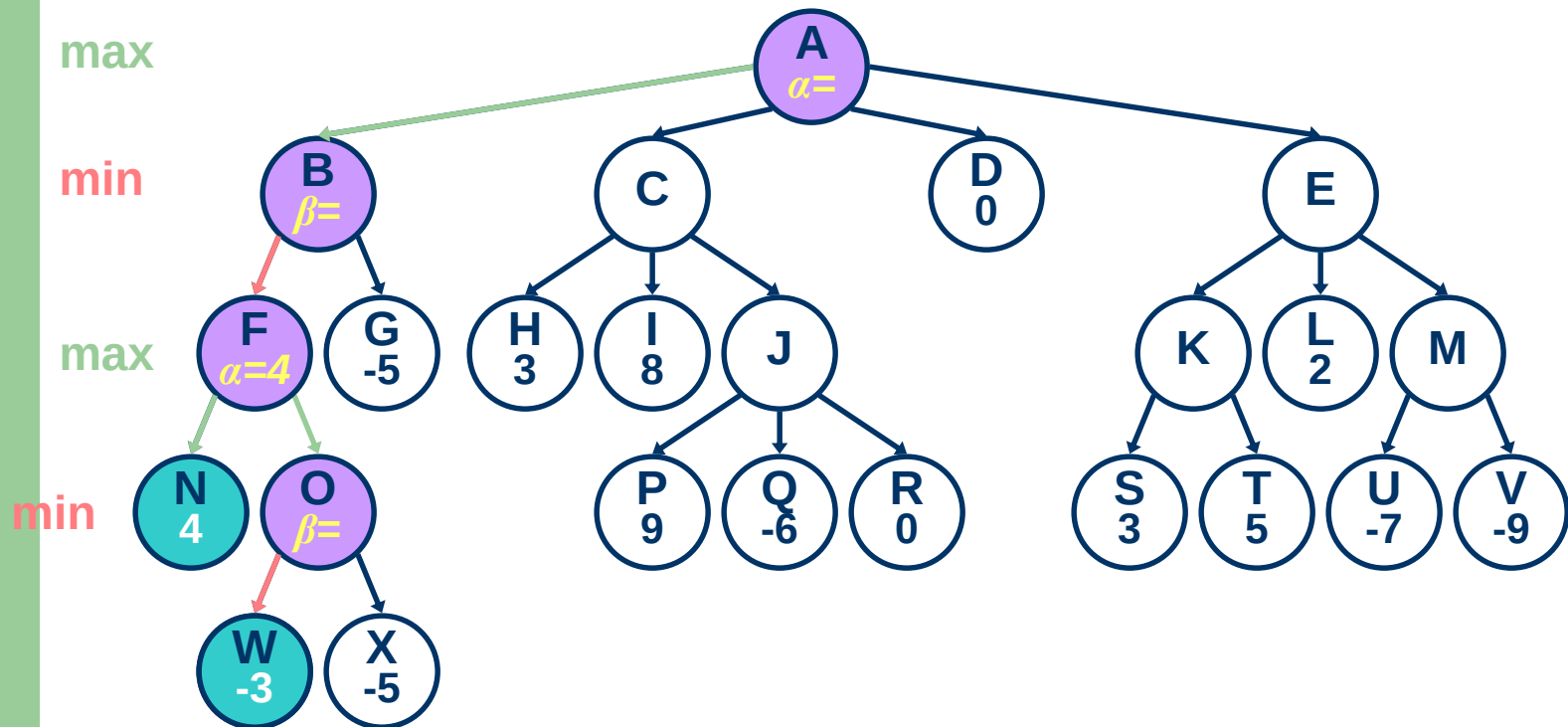


Call
Stack

O
F
B
A

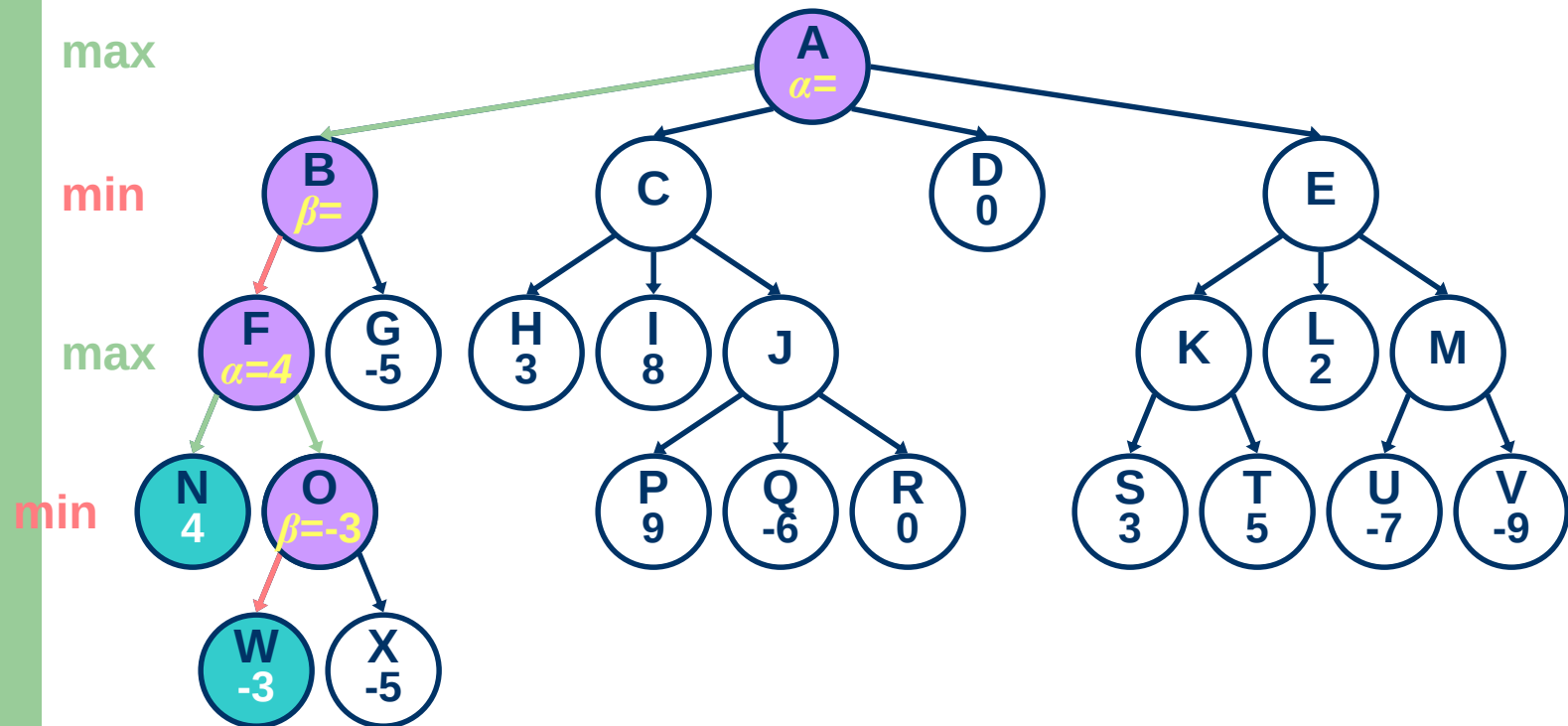
Ví dụ

`minimax (W, 4, 4)`



Ví dụ

`minimax(0, 3, 4)`
`beta = -3`



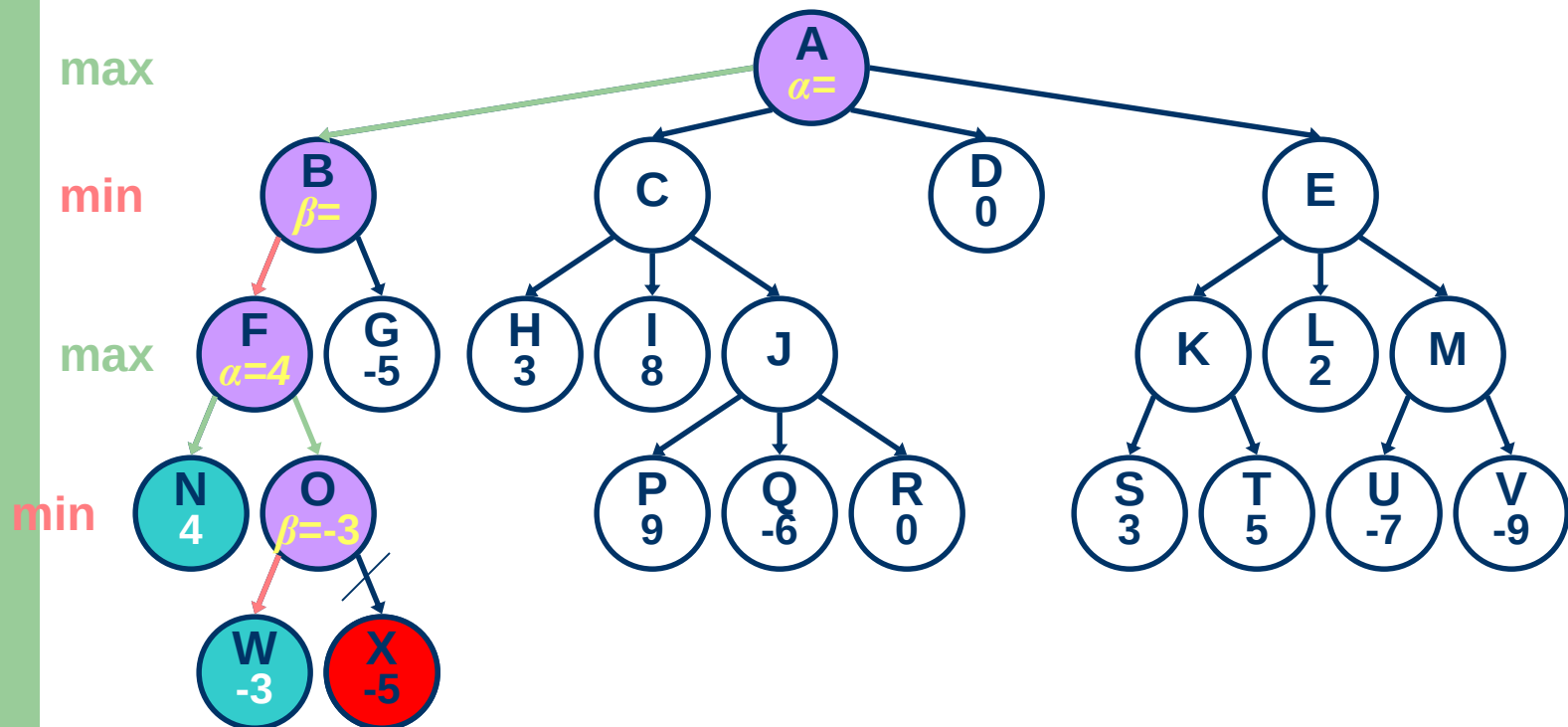
Call
Stack

O
F
B
A

Ví dụ

$\text{minimax}(0, 3, 4)$

$\text{beta}(\text{O}) \leq \text{alpha}(\text{F})$: (alpha cut-off)

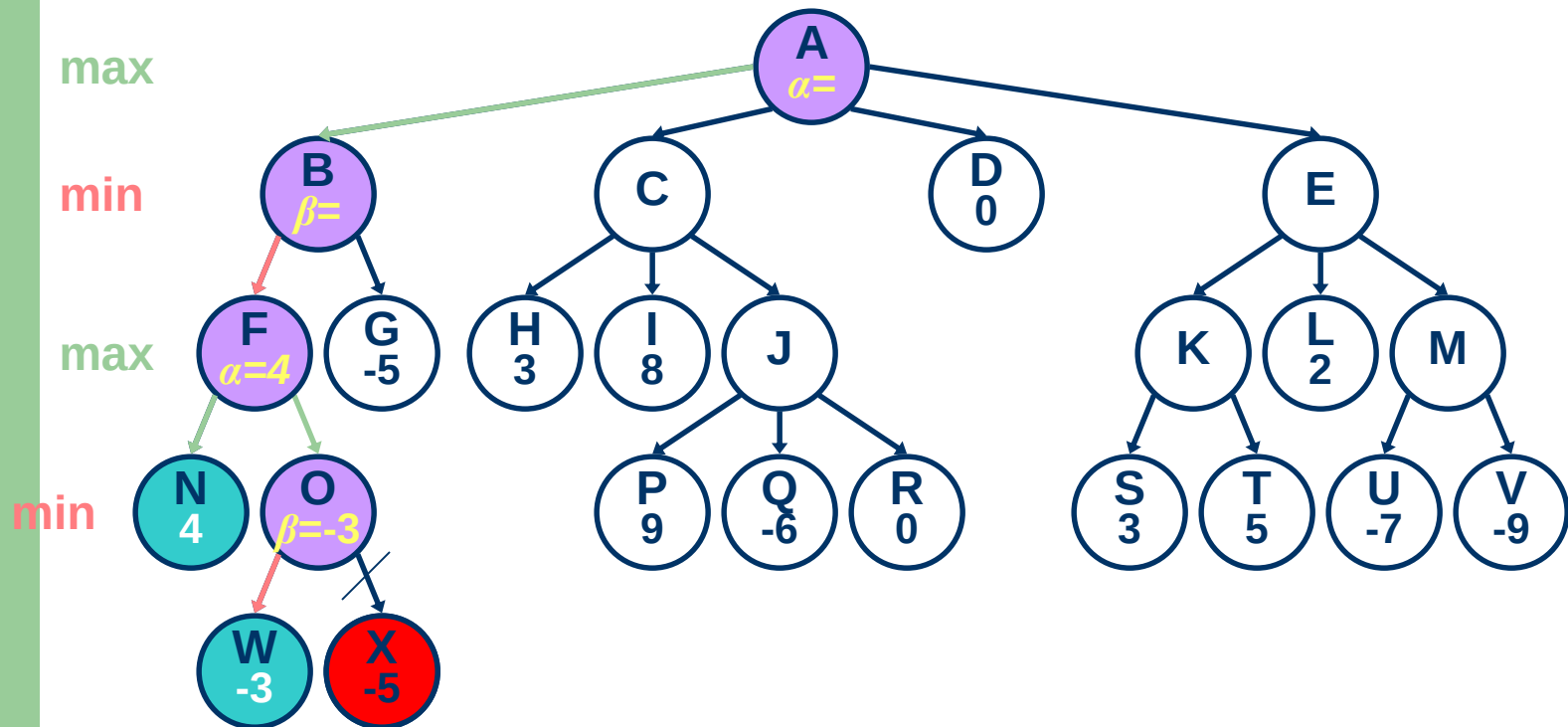


Call
Stack

O
F
B
A

Ví dụ

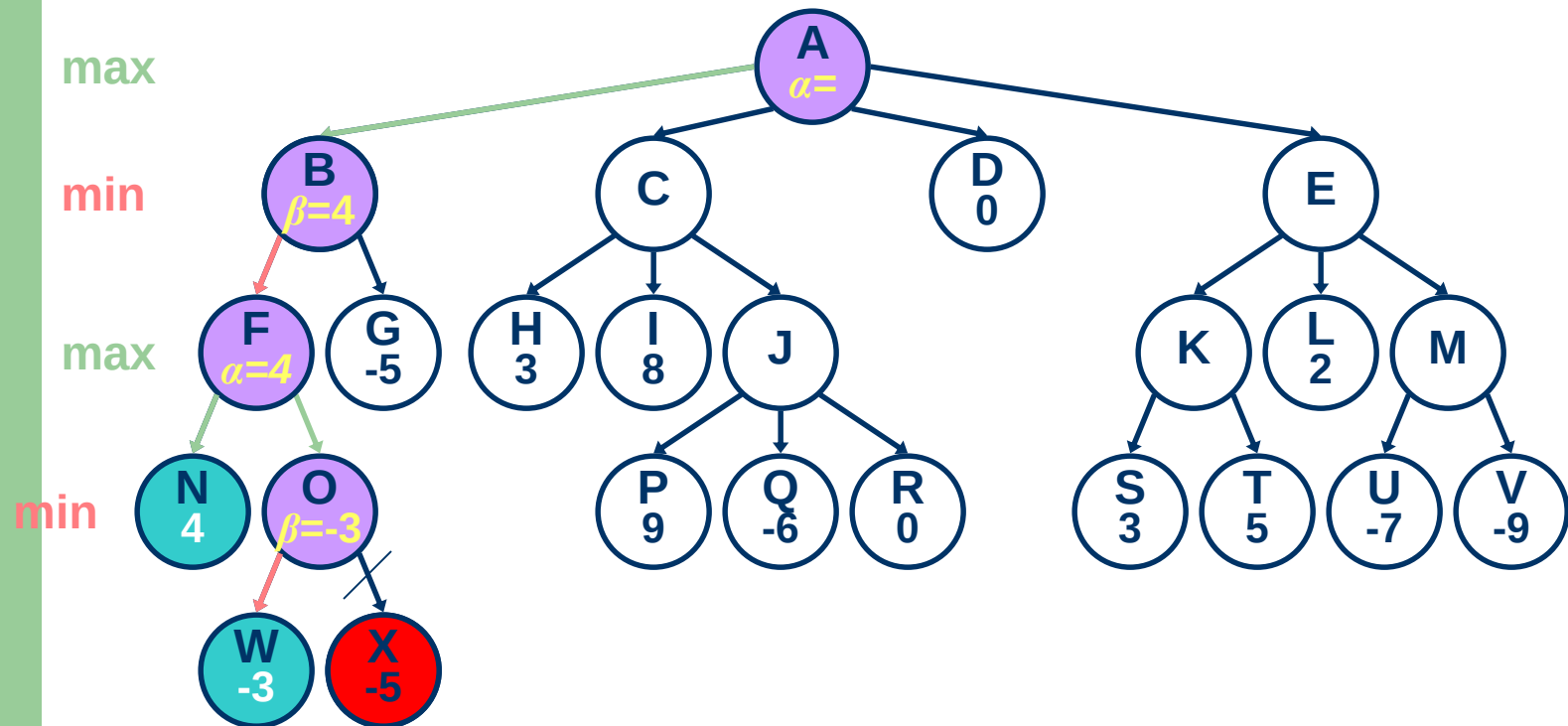
`minimax(F, 2, 4)`



Ví dụ

`minimax(B, 1, 4)`

`beta = 4`

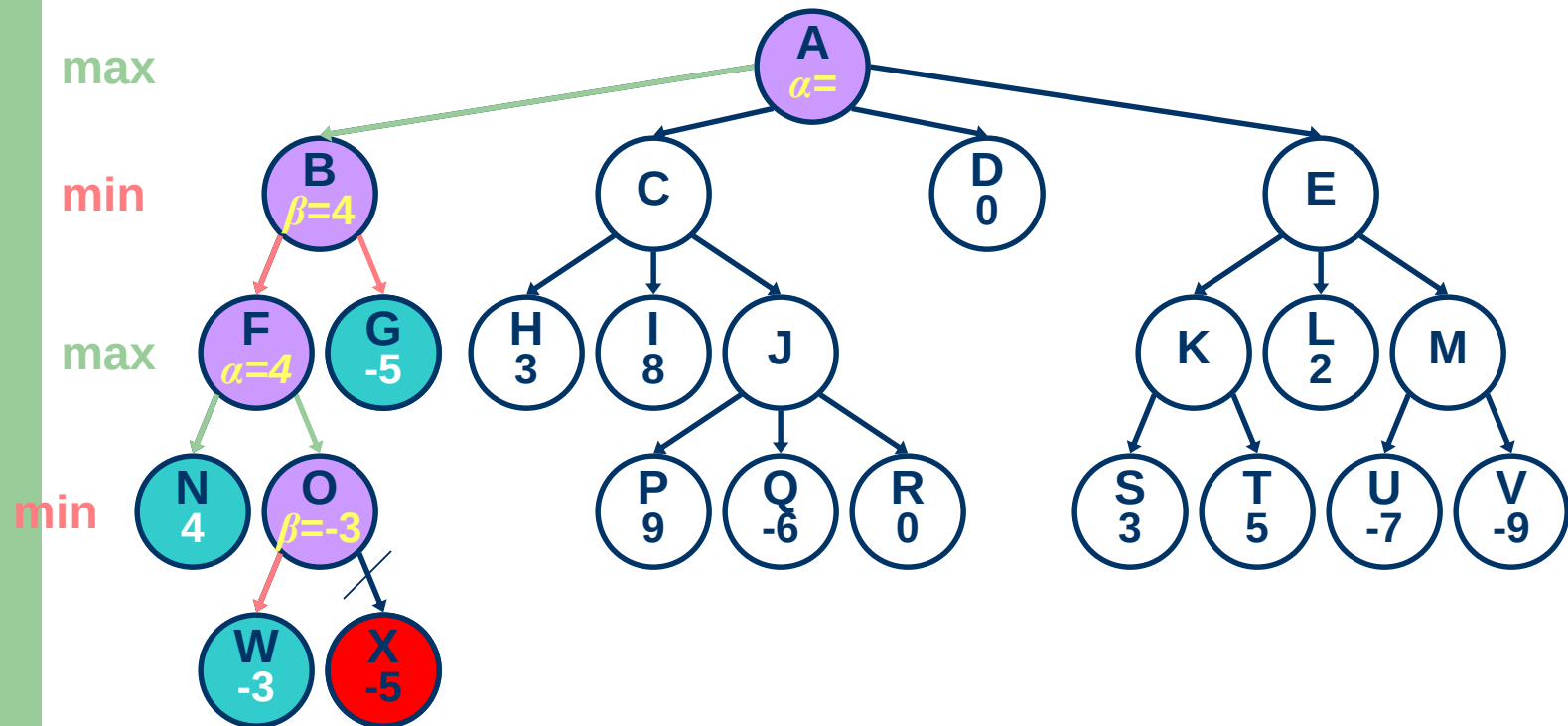


Call
Stack

B
A

Ví dụ

`minimax(G, 2, 4)`



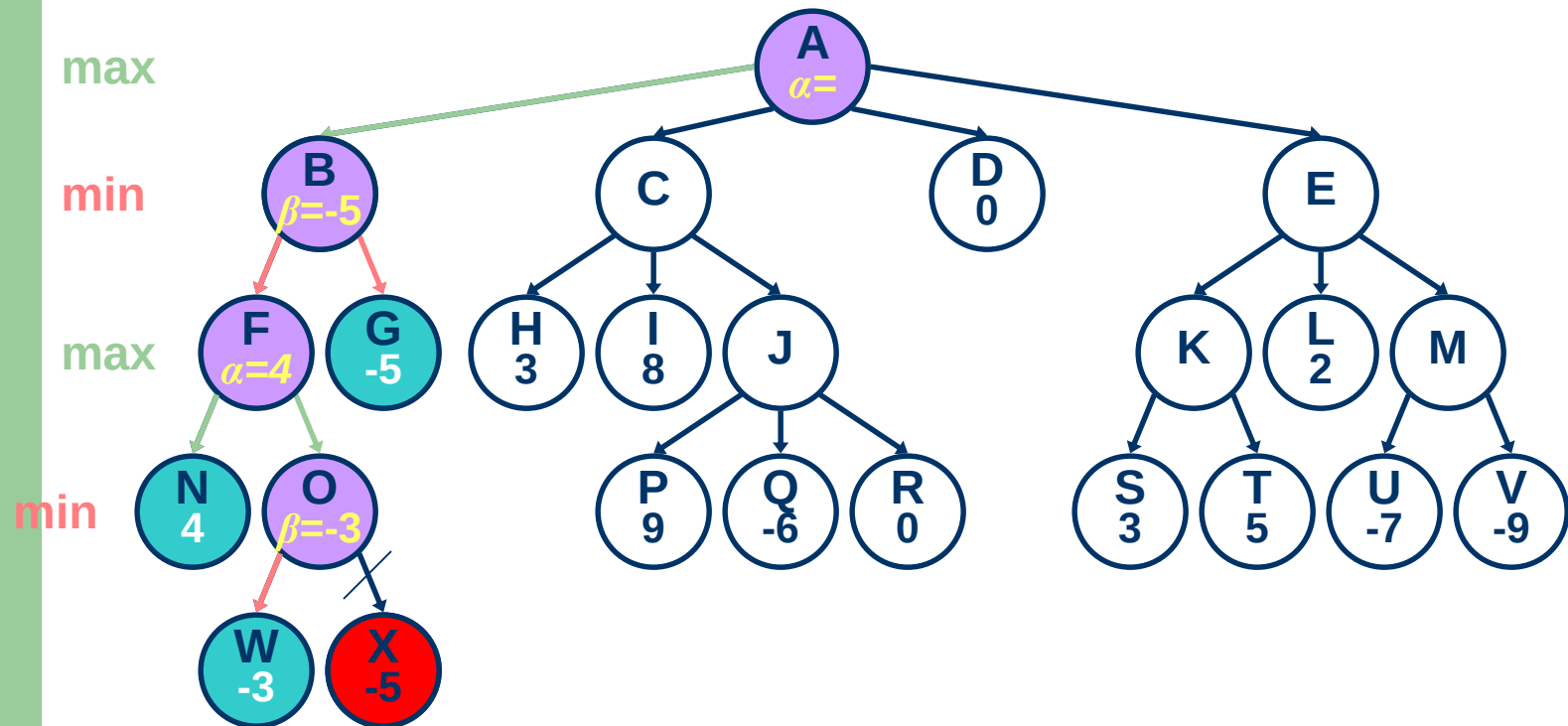
Call
Stack

G
B
A

Ví dụ

`minimax(B, 1, 4)`

`beta = -5`



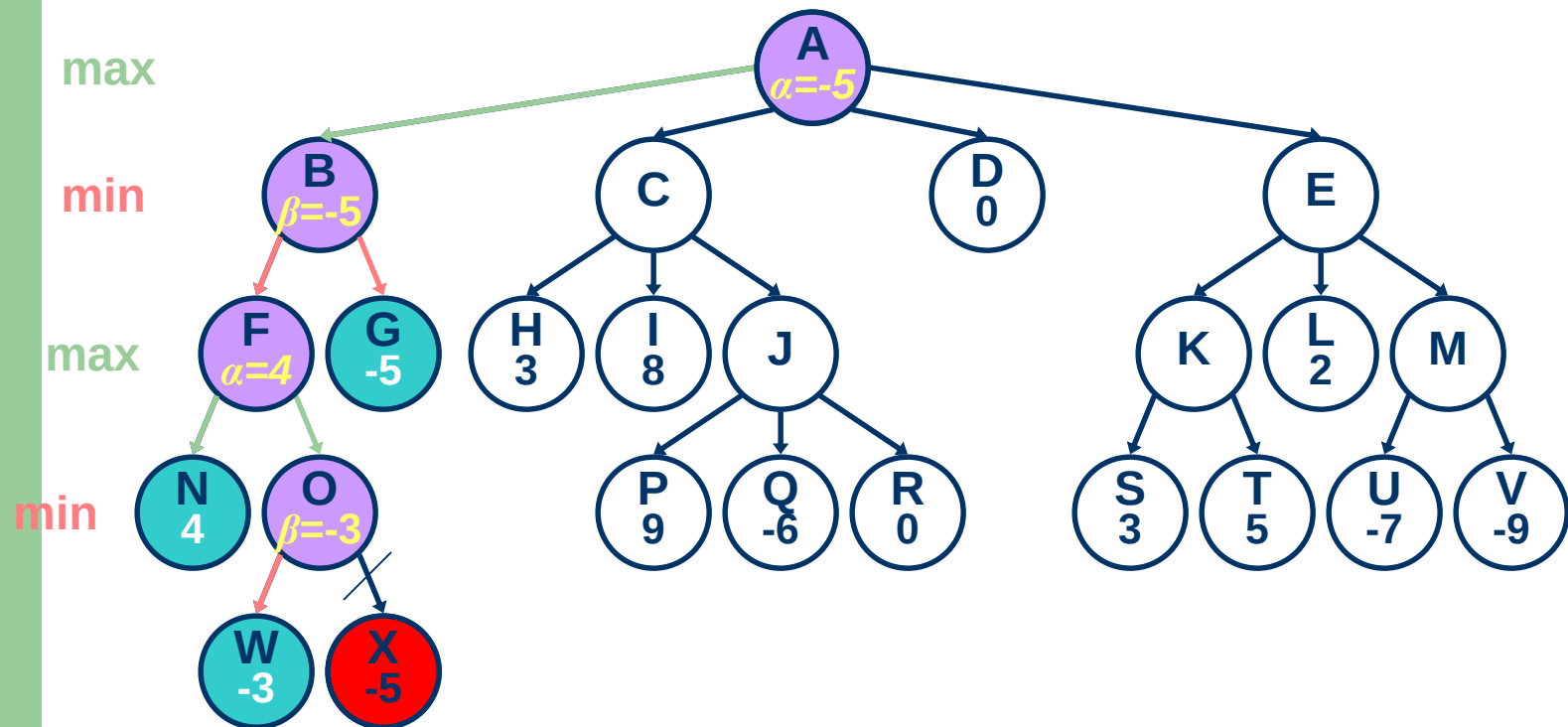
Call
Stack

B
A

Ví dụ

`minimax(A, 0, 4)`

`alpha = -5`

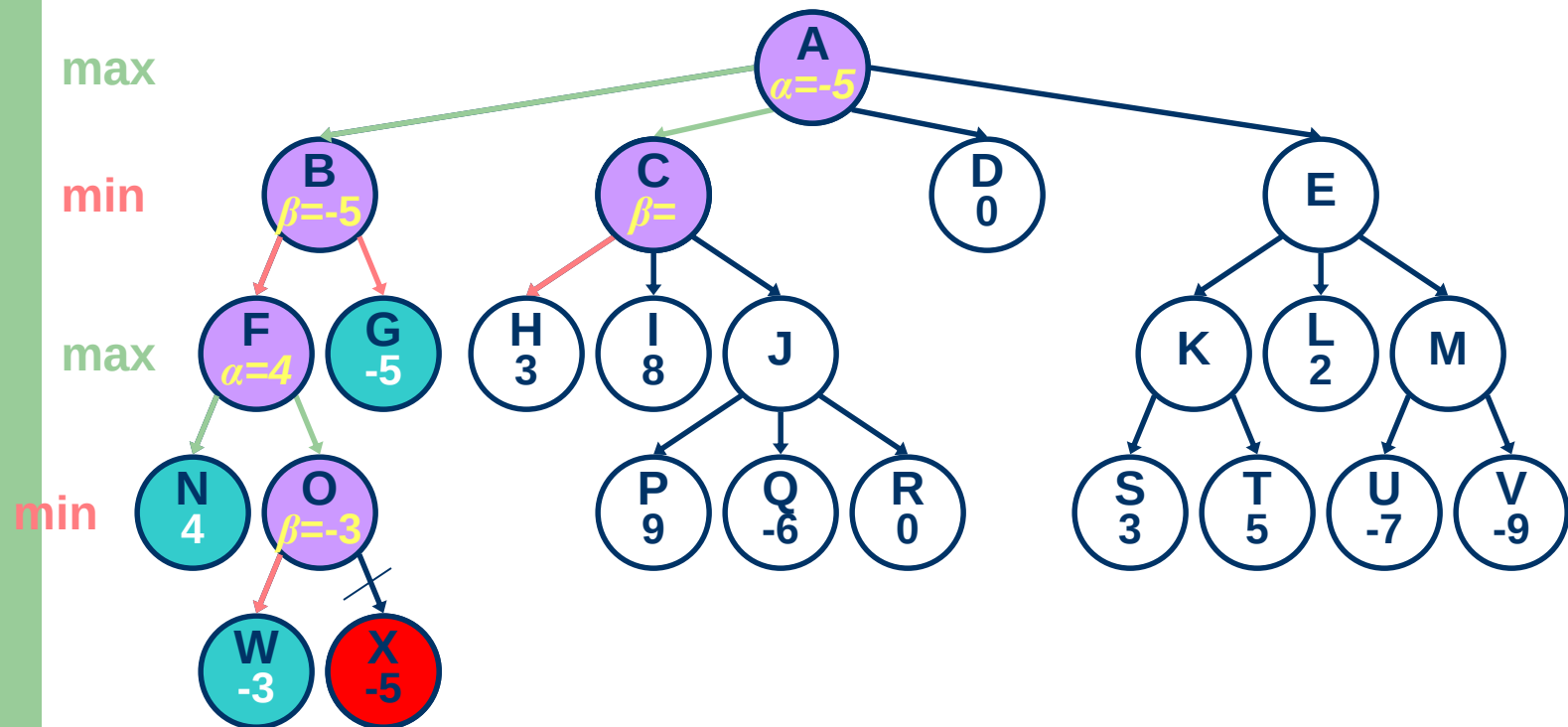


Call
Stack

A

Ví dụ

`minimax(C, 1, 4)`

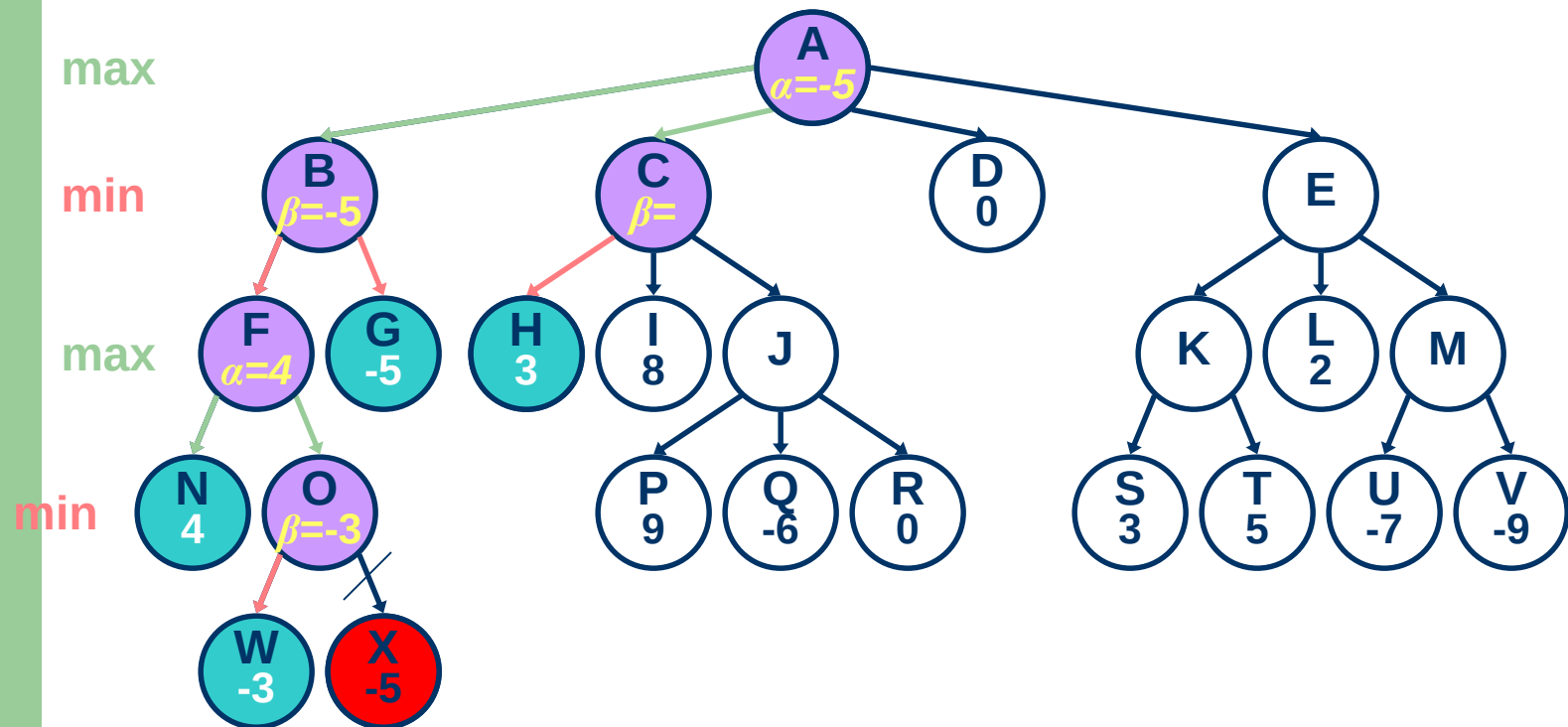


Call
Stack

C
A

Ví dụ

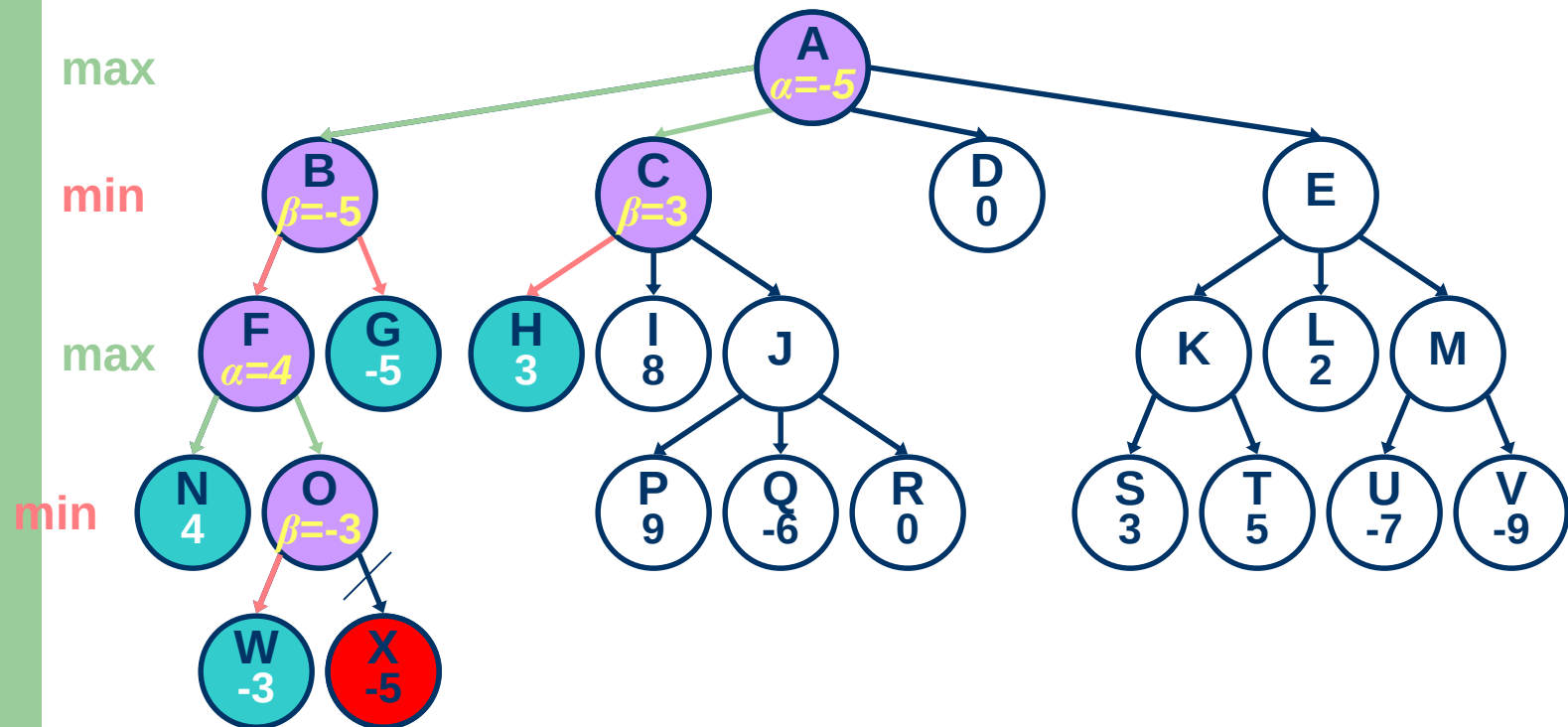
`minimax(H, 2, 4)`



Ví dụ

`minimax(C, 1, 4)`

`beta = 3`

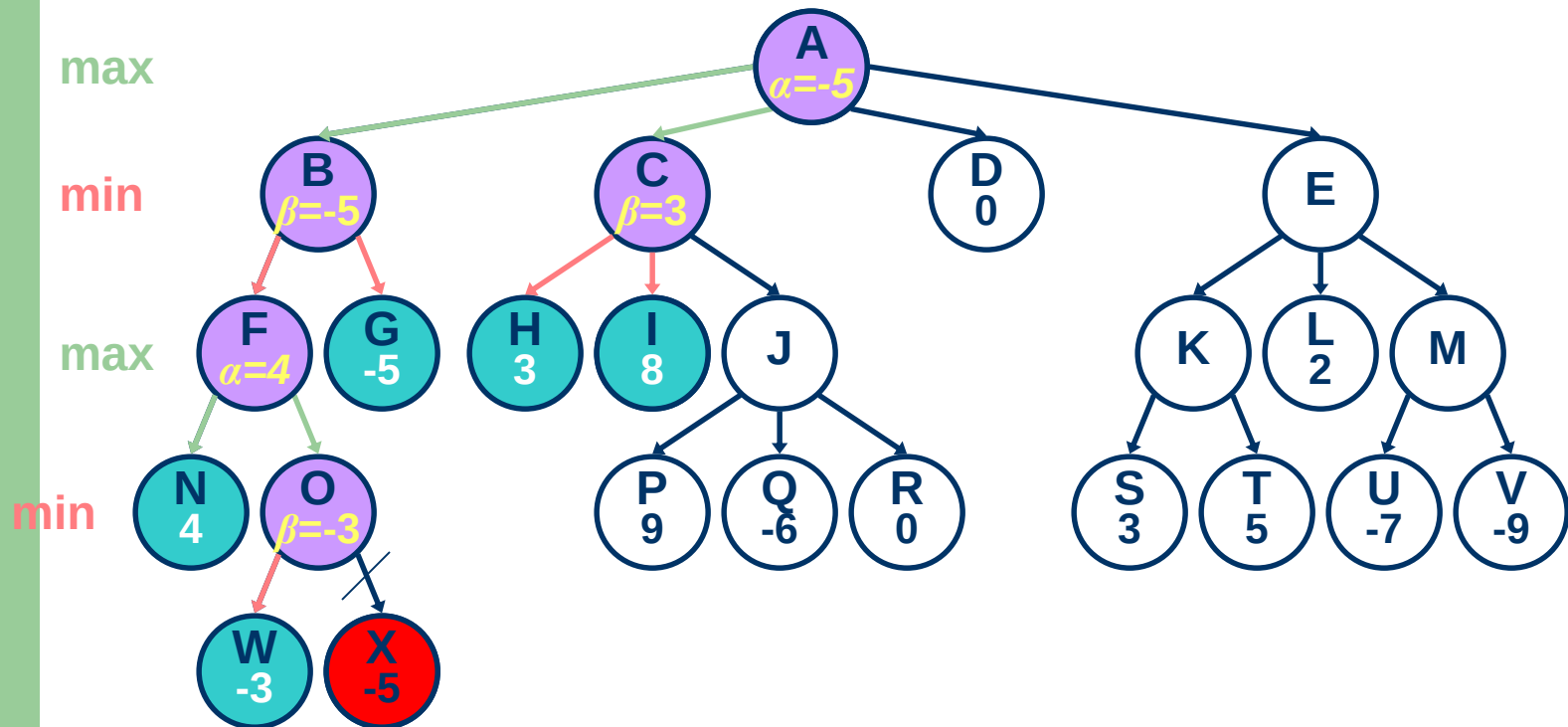


Call
Stack

C
A

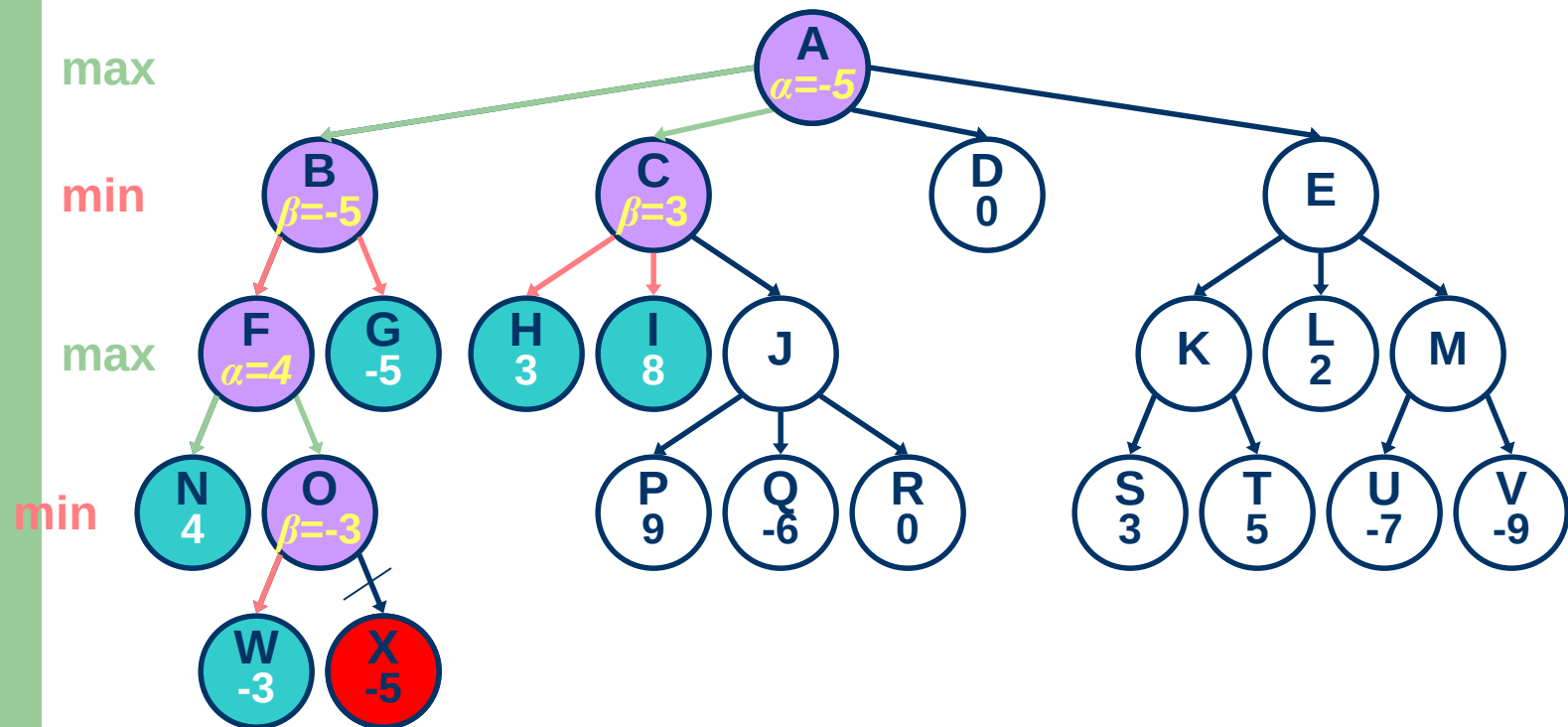
Ví dụ

`minimax(I, 2, 4)`



Ví dụ

`minimax (C, 1, 4)`

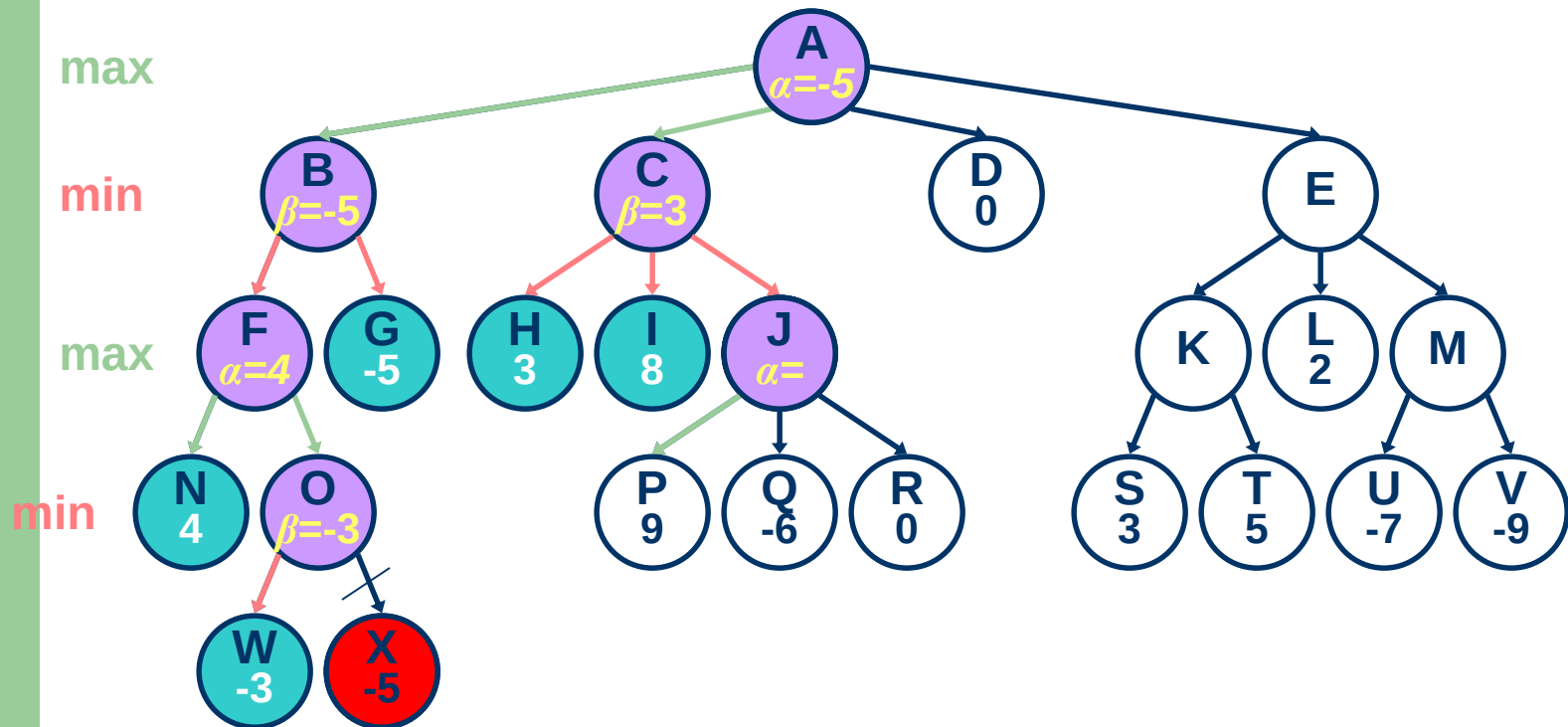


Call
Stack

C
A

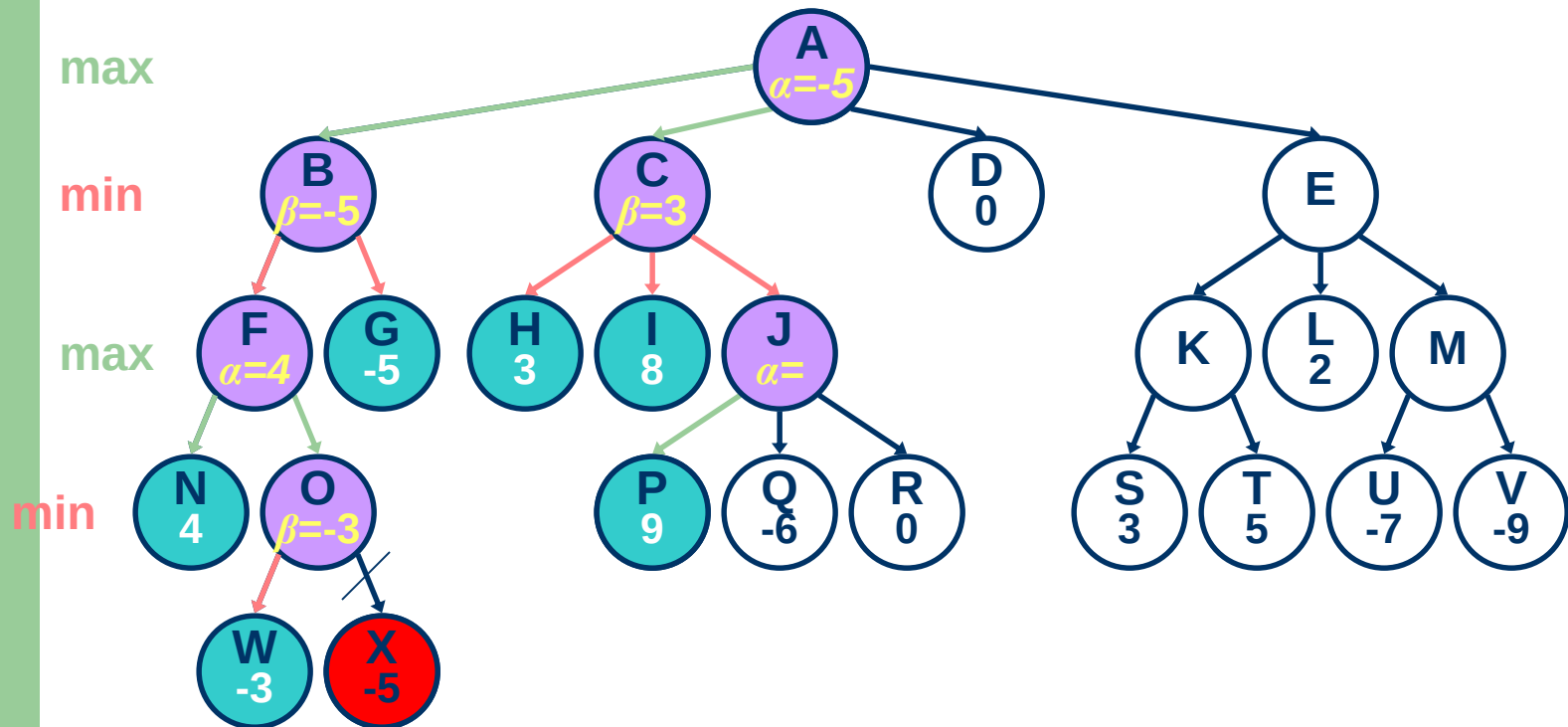
Ví dụ

`minimax(J, 2, 4)`



Ví dụ

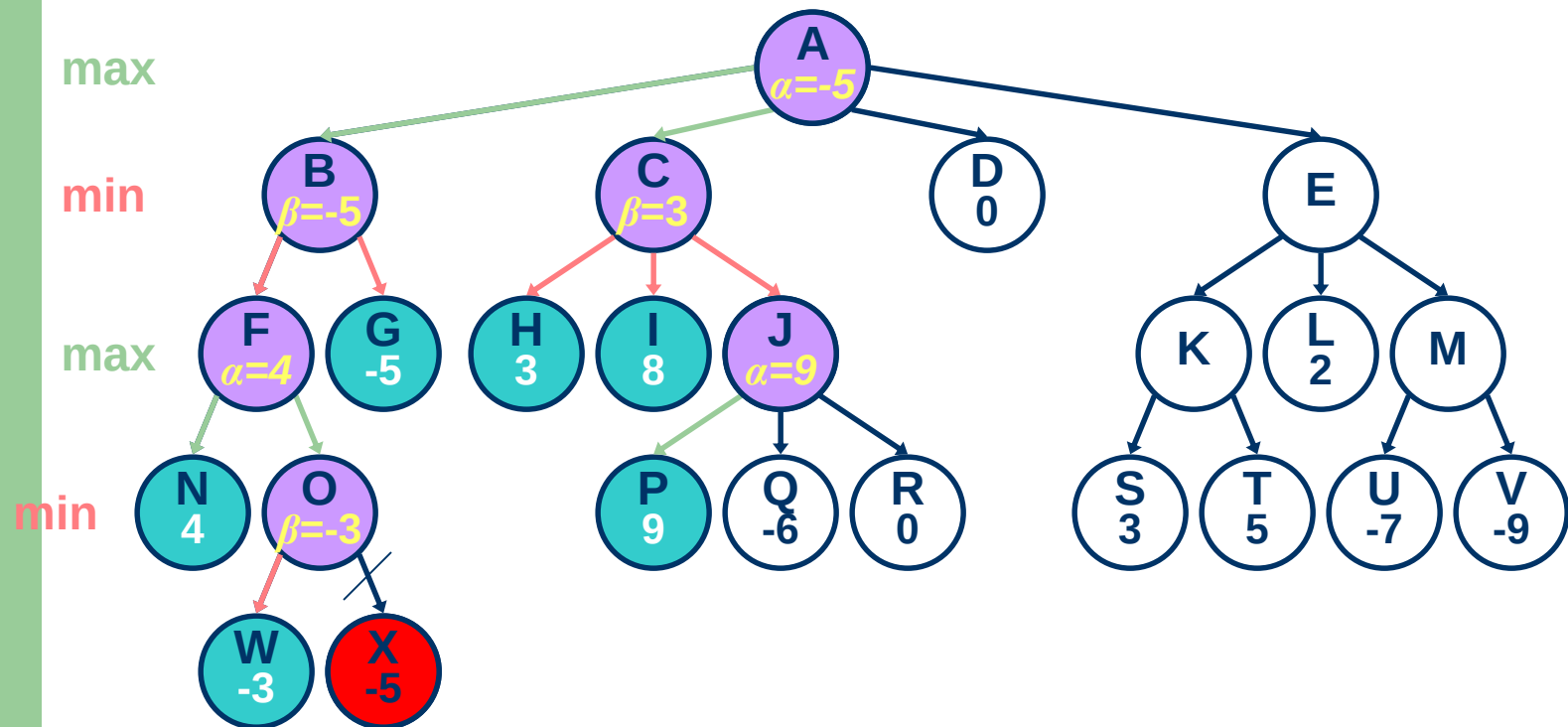
`minimax(P, 3, 4)`



Ví dụ

`minimax(J, 2, 4)`

`alpha = 9`



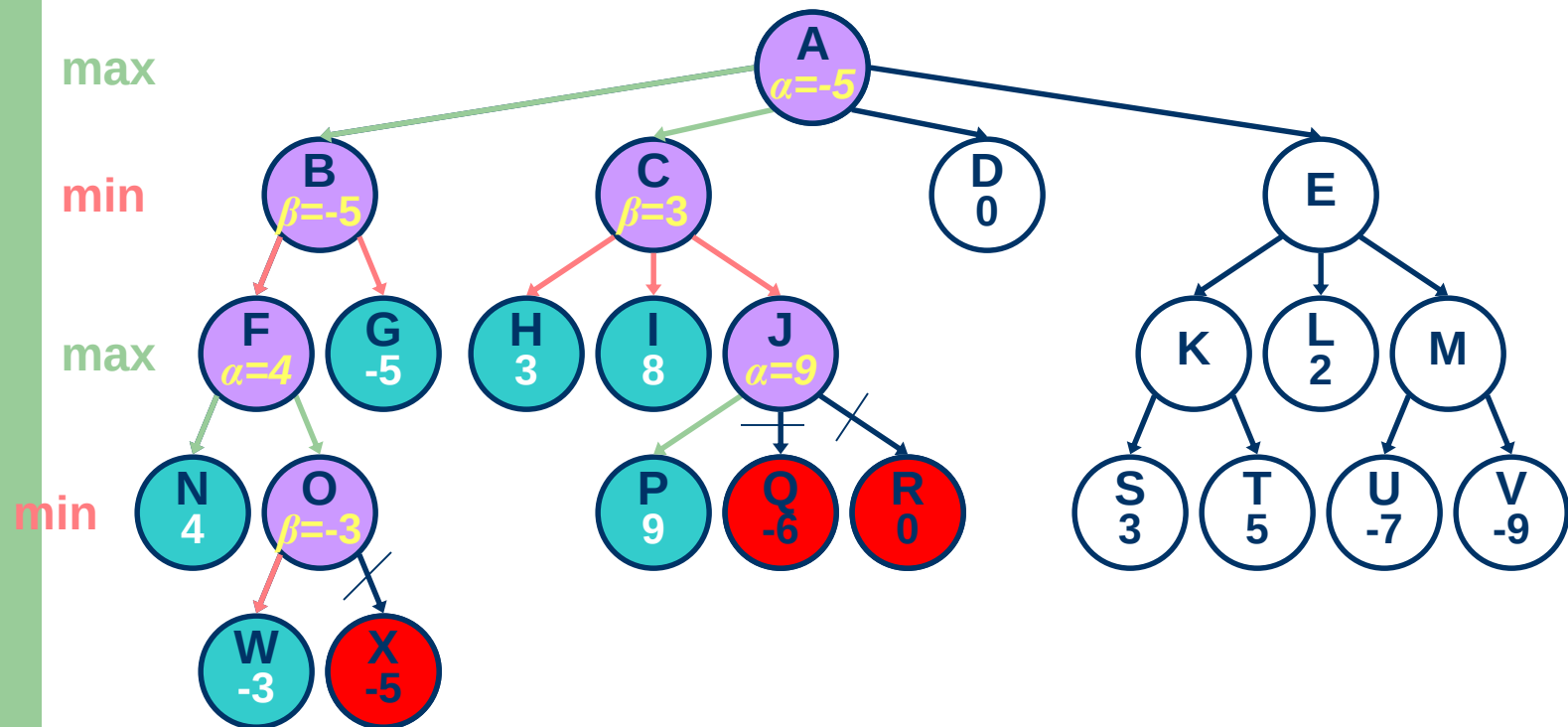
Call
Stack

J
C
A

Ví dụ

`minimax(J, 2, 4)`

$\alpha(J) \geq \beta(C)$: (beta cut-off)

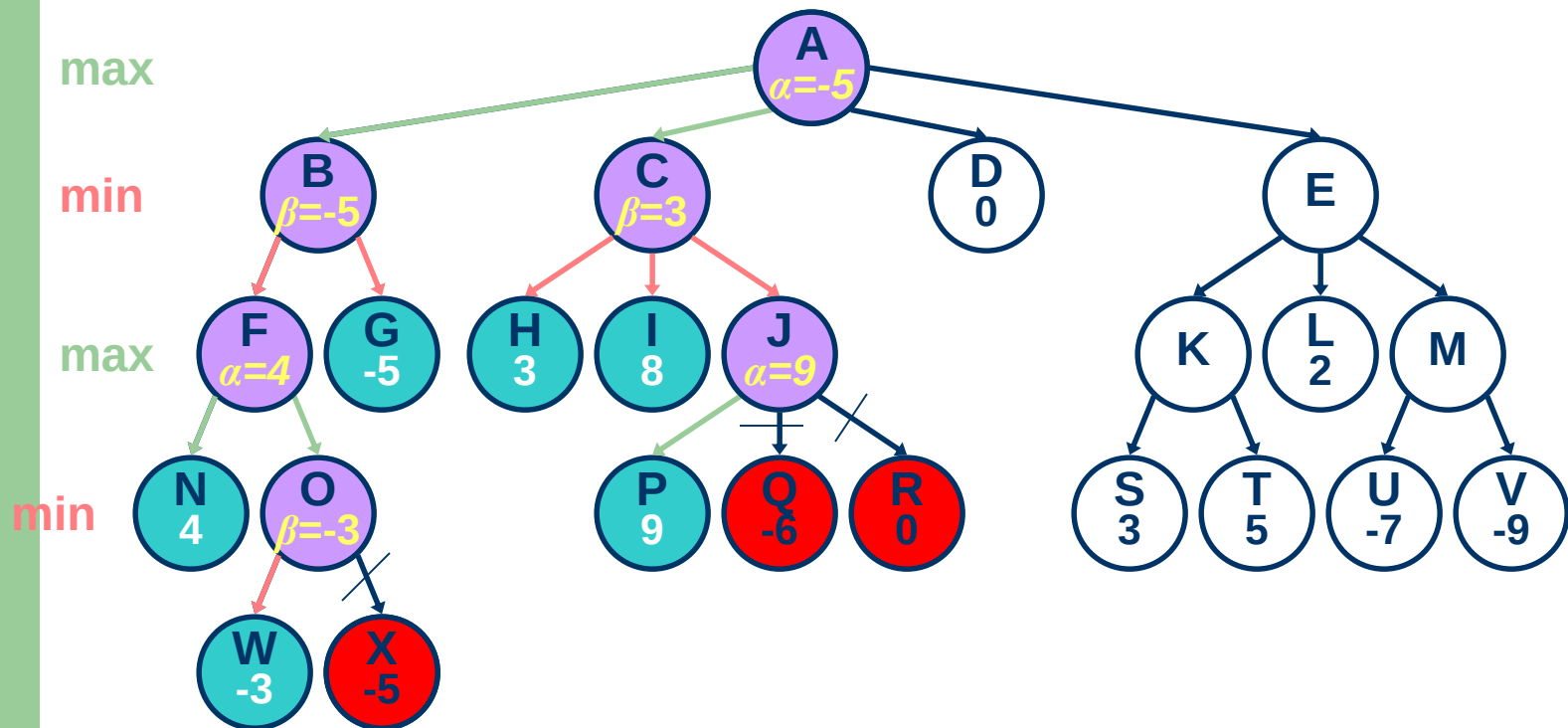


Call
Stack

J
C
A

Ví dụ

`minimax (C, 1, 4)`



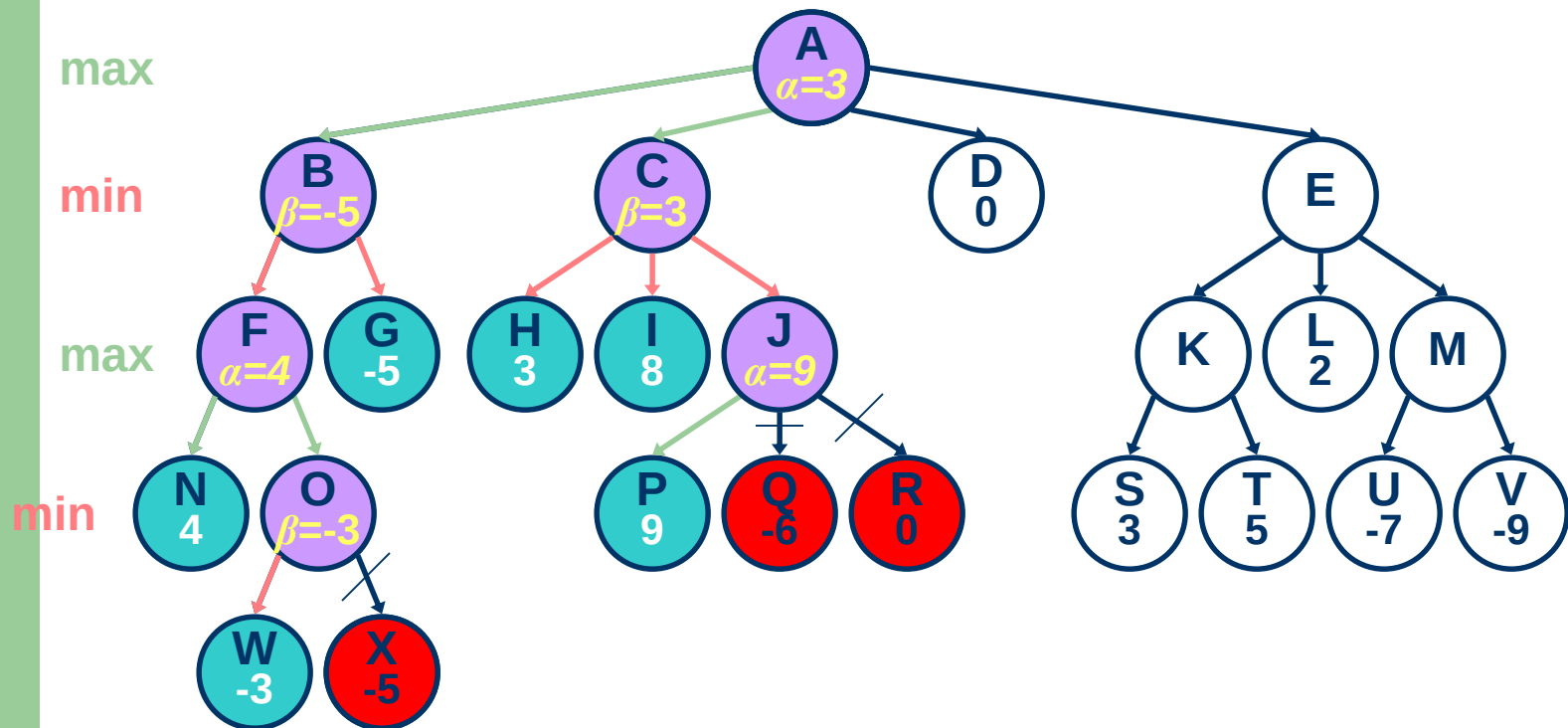
Call
Stack

C
A

Ví dụ

`minimax(A, 0, 4)`

`alpha = 3`

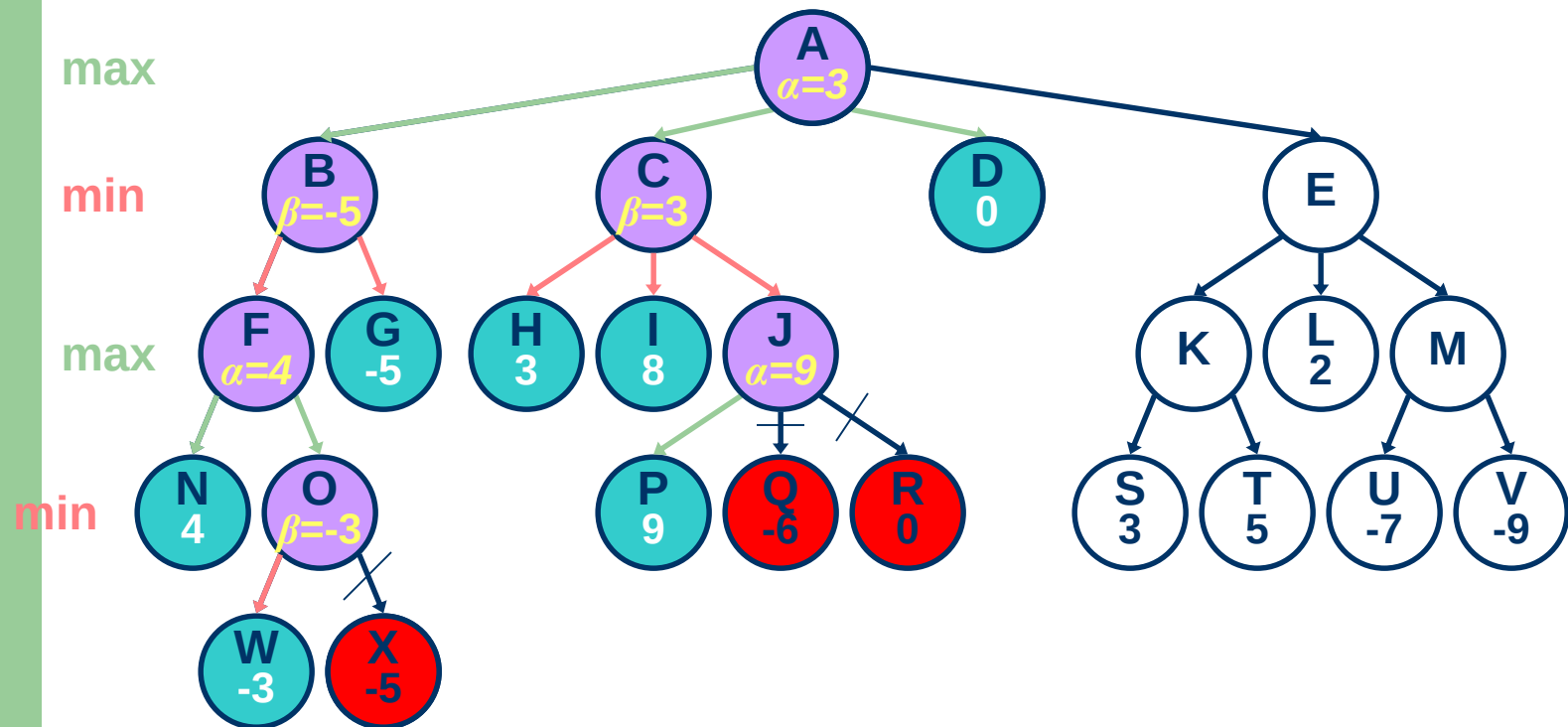


Call
Stack

A

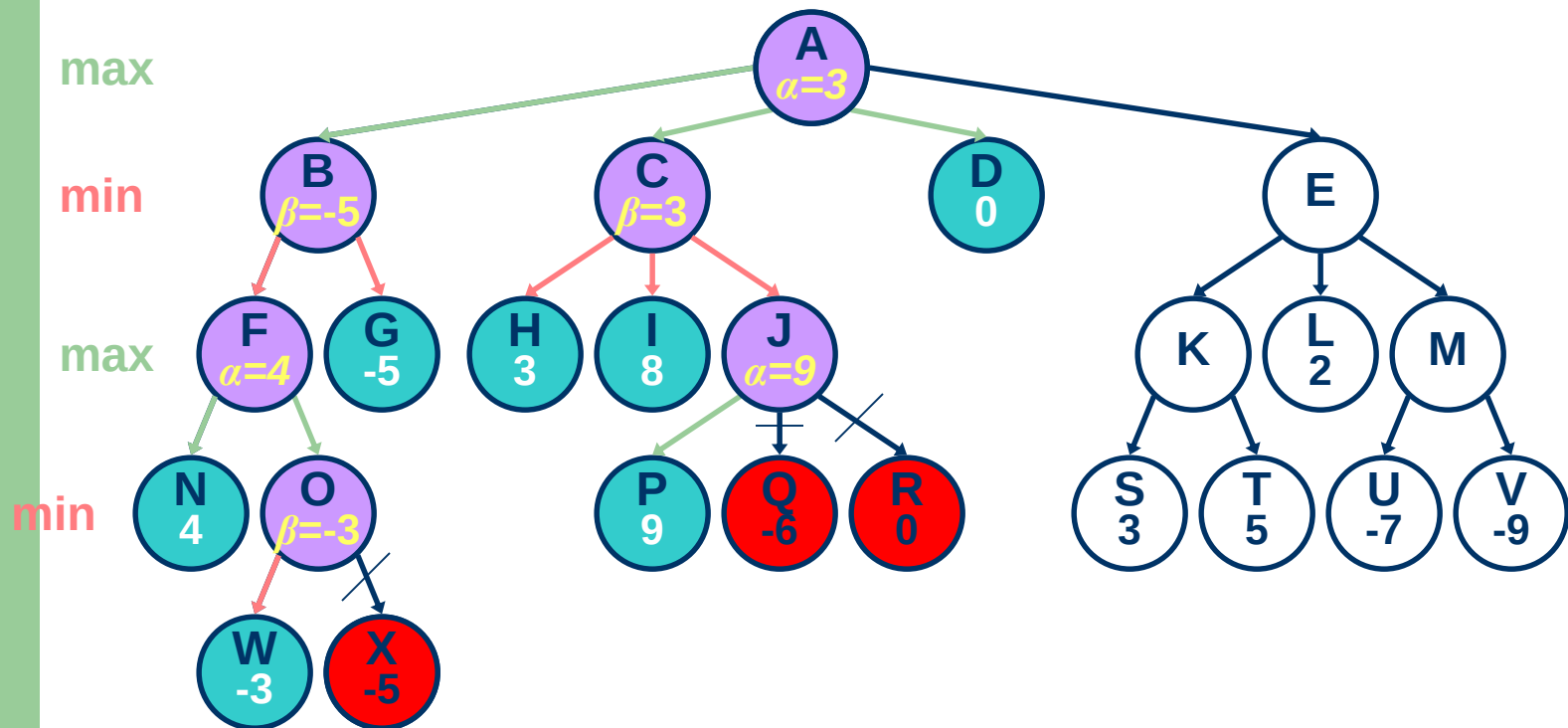
Ví dụ

`minimax(D, 1, 4)`



Ví dụ

`minimax(A, 0, 4)`

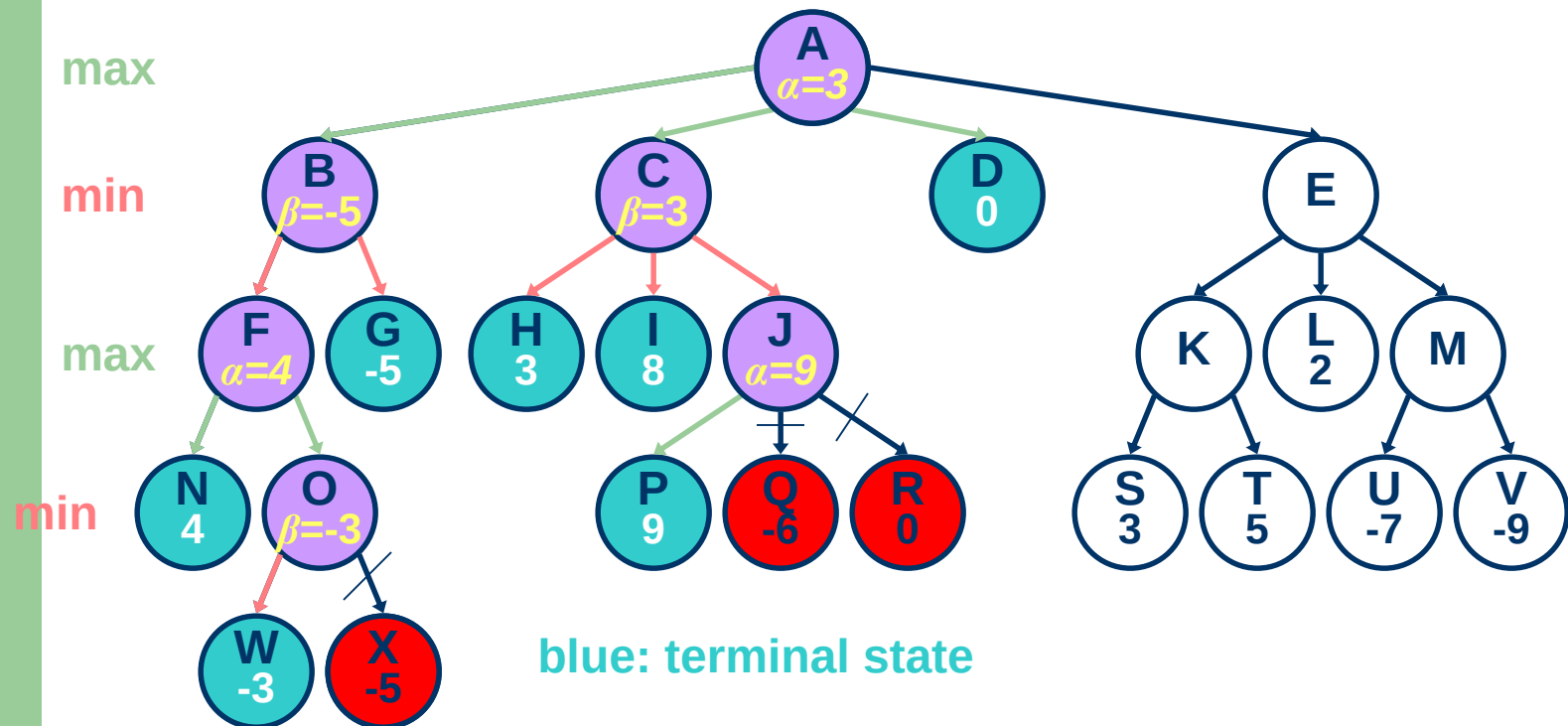


Call
Stack

A

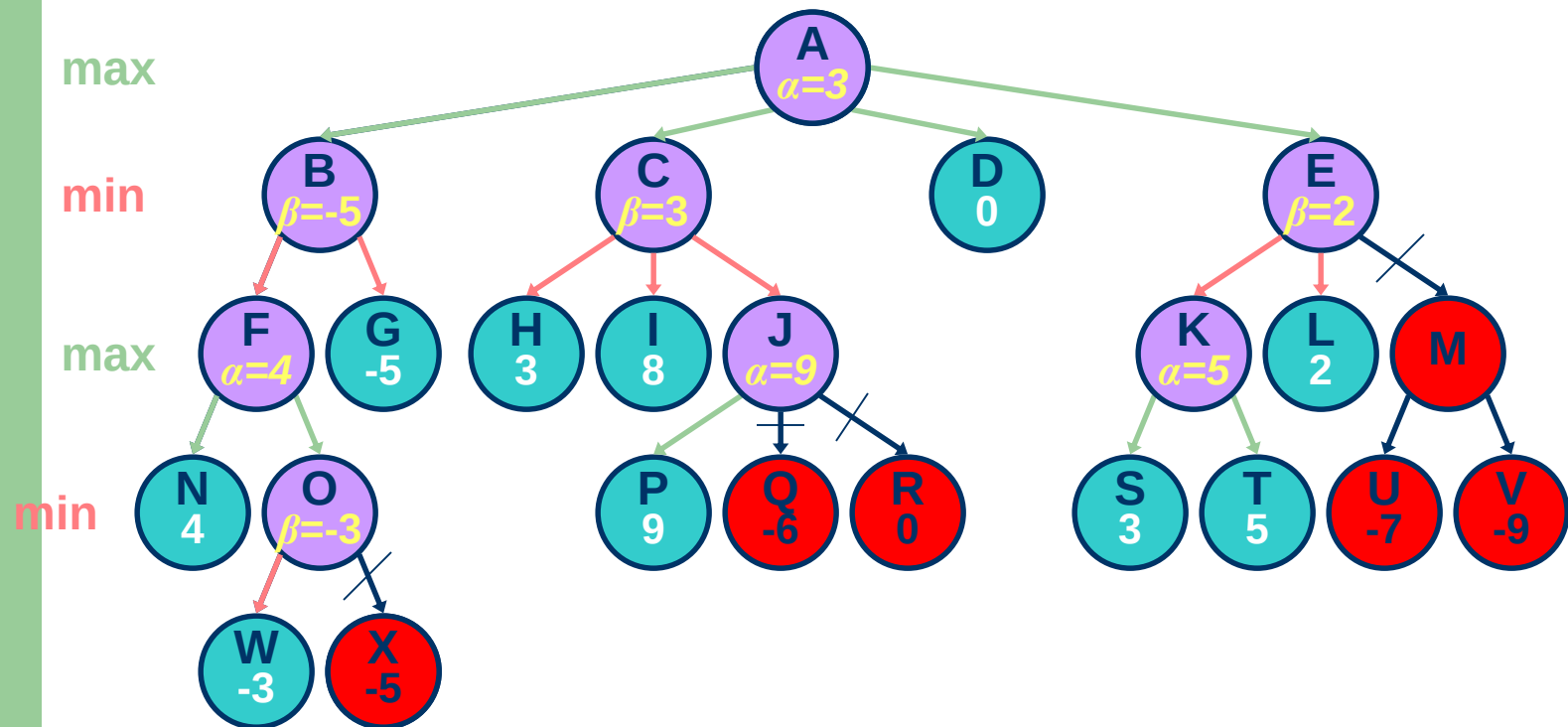
Ví dụ

How does the algorithm finish the search tree?



Ví dụ

beta(E) ≤ alpha(A): (alpha cut-off)

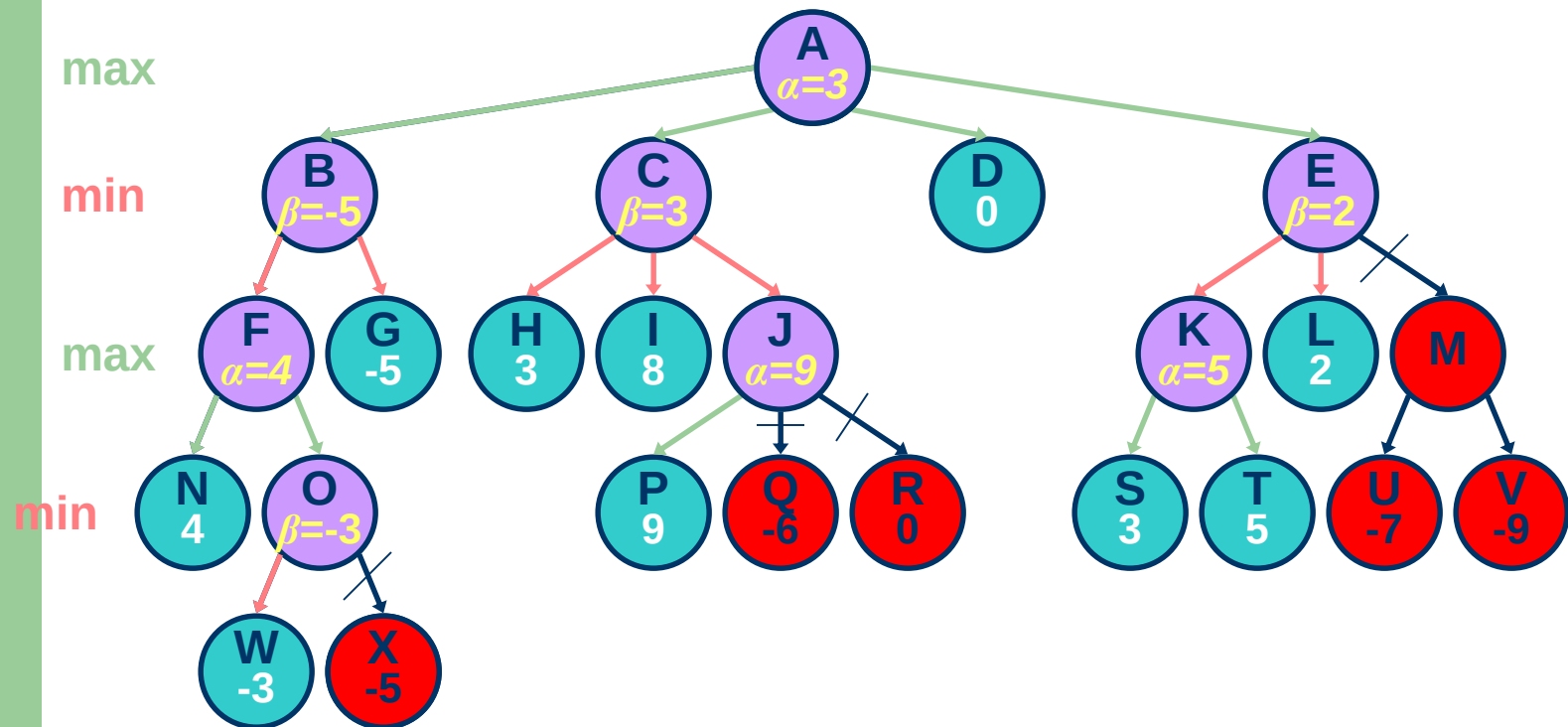


Call
Stack

A

Ví dụ

Kết quả : Chọn nước đi C



Hiệu quả của phép tỉa Alpha-Beta

- * *Hiệu quả phụ thuộc thứ tự các nút được lựa chọn, nếu nút tốt nhất luôn được lựa chọn trước thì các nút còn lại đều bị tỉa*
- **Trường hợp tồi nhất:**
 - Không có phép tỉa
 - Tìm kiếm vét cạn
- **Trường hợp tốt nhất:**
 - Nút tốt được đánh giá trước

Hiệu quả của phép tỉa Alpha-Beta

- Thực tế thấy rằng, độ phức tạp thường là $O(b^{(d/2)})$ so với $O(b^d)$ của giải thuật *MINIMAX* tổng quát
- Ví dụ cờ vua
 - $b \sim 35 \rightarrow b \sim 6$
 - Có thể tìm kiếm sâu hơn

Giới hạn thời gian

- Thực tế, nhiều trò chơi đòi hỏi phải giới hạn thời gian T cho mỗi nước đi
- Chiến lược?
 - Không thể dùng giải thuật Alpha-Beta
 - Đặt độ sâu vừa đủ để đáp ứng thời gian T
 - Có thể bỏ lỡ những nước đi tốt

Giới hạn thời gian

- Trong thực tiễn, giải thuật tìm kiếm sâu dần IDS có thể sử dụng
 - Thực hiện alpha-beta với độ sâu tăng dần
 - Khi hết thời gian, kết quả trả về là kết quả của vòng cuối cùng

Hiệu ứng chân trời

- Đôi khi nước được chọn rất tốt cho một độ sâu nhất định nhưng lại để lại hậu quả xấu cho các nước tiếp theo
 - Máy tính bắt quân hậu nhưng vài nước sau bị chiếu hết
- Máy tính có khả năng quan sát hạn chế (**limited horizon**); Nó không có khả năng dự báo tiếp các nước tiếp theo
- Giải pháp?
 - Tìm kiếm ngằm (quiescence search)
 - Tìm kiếm phụ (secondary search)

Hiệu ứng chân trời

- **Tìm kiếm ngàm**
 - Tìm kiếm sâu hơn độ sâu lựa chọn
- **Tìm kiếm phụ**
 1. Tìm nước đi tốt nhất với độ sâu d
 2. Kiểm tra k bước tiếp để khẳng định đó là nước đi chấp nhận
 3. Nếu không được, tìm khả năng tiếp theo

Sử dụng thư viện nước đi

- Xây dựng thư viện các nước đi cho khai cuộc, tàn cuộc và một số nước đi đặc biệt
- Khi thực hiện một nước đi, kiểm tra thấy trong cơ sở dữ liệu, sử dụng cơ sở dữ liệu:
 - Để xác định nước đi tiếp theo
 - Đánh giá lợi thế của nước đi
- Nếu không tìm thấy, áp dụng giải thuật Alpha-beta

Hàm giá

* ***Là hàm heuristic đánh giá lợi thế của nước đi***

- $\text{cost}(f_1, f_2, \dots, f_n)$
- f_i : là các thông số về trạng thái bàn cờ
 - f_1 , số quân trắng
 - f_2 , số quân đen
 - $f_3, f_1/f_2$
 - f_4 , đánh giá sự đe dọa với vua trắng
 - ...

Hàm giá tuyến tính

- $\text{cost}(f_1, f_2, \dots, f_n) = \sum w_i \cdot f_i$
 - w_i : các trọng số
 - f_i : các thông số

*** Thông số quan trọng sẽ có trọng số cao**

Hàm giá

- * *Chất lượng người chơi phụ thuộc trực tiếp vào hàm giá*
- **Để xây dựng hàm giá tốt:**
 1. Lựa chọn tốt các thông số (chuyên gia)
 2. Lựa chọn tốt các trọng số

Xây dựng bộ trọng số bằng cách học

- **Ý tưởng:**

Chơi nhiều lần với một đối thủ nào đó

- Với mỗi phép dịch chuyển (hoặc cả trò chơi), tính giá trị

$\text{error} = \text{đầu ra thực tế (kết quả thực của phép dịch chuyển)} - \text{hàm giá}$

- Nếu $\text{error} > 0$,
Hiệu chỉnh trọng số để làm tăng hàm giá

- $\text{error} < 0$
Hiệu chỉnh trọng số để làm giảm hàm giá

Ví dụ

Chơi cờ:

A. L. Samuel, “Some Studies in Machine Learning using the Game of Checkers,” *IBM Journal of Research and Development*, **11**(6):601-617, 1959

- Học bằng cách chơi hàng nghìn lần
- Chỉ sử dụng IBM 704 với 10K RAM, magnetic tape, tốc độ 1 kHz
- Đủ cạnh tranh với đối thủ bình thường

Ví dụ

Backgammon:

G. Tesauro and T. J. Sejnowski, “A Parallel Network that Learns to Play Backgammon,” *Artificial Intelligence* **39**(3), 357-390, 1989

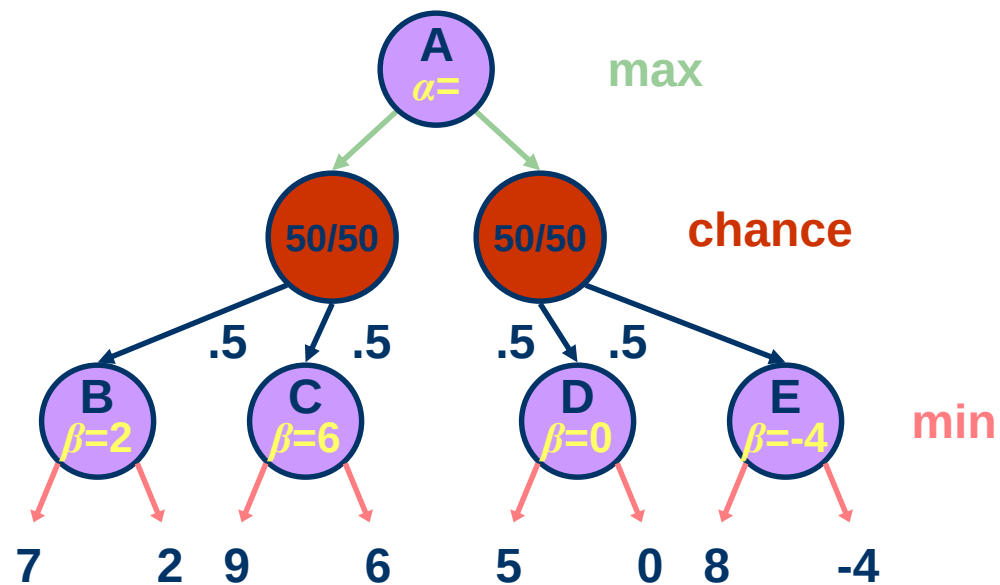
- Cũng học bằng cách chơi nhiều lần
- Sử dụng hàm giá phi tuyến - mạng nơ ron
- Được đánh giá là một trong 3 trò chơi nổi tiếng

Những trò chơi may rủi (non-deterministic)

- **Một số trò chơi với may rủi:**
 - Roulette
 - Xúc xắc
 - Bài lá
- **Làm thế nào xử lý yếu tố ngẫu nhiên?**
- **Cây trò chơi sẽ mở rộng với các nút đặc biệt : nút cơ hội (chance nodes):**
 1. Máy tính quyết định
 2. Ngẫu nhiên
 3. Đối phương quyết định

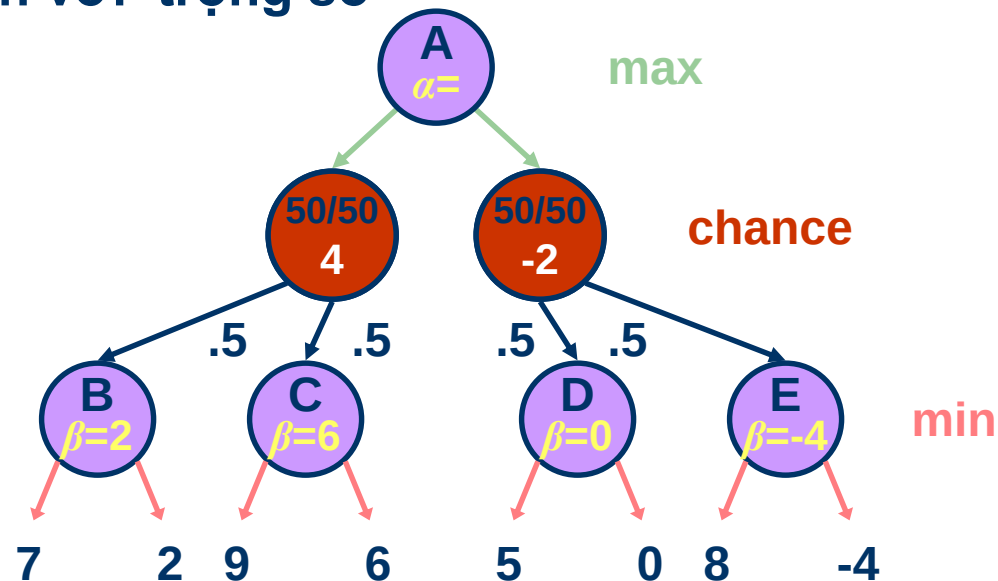
Trò chơi may rủi

Cây trò chơi mở rộng



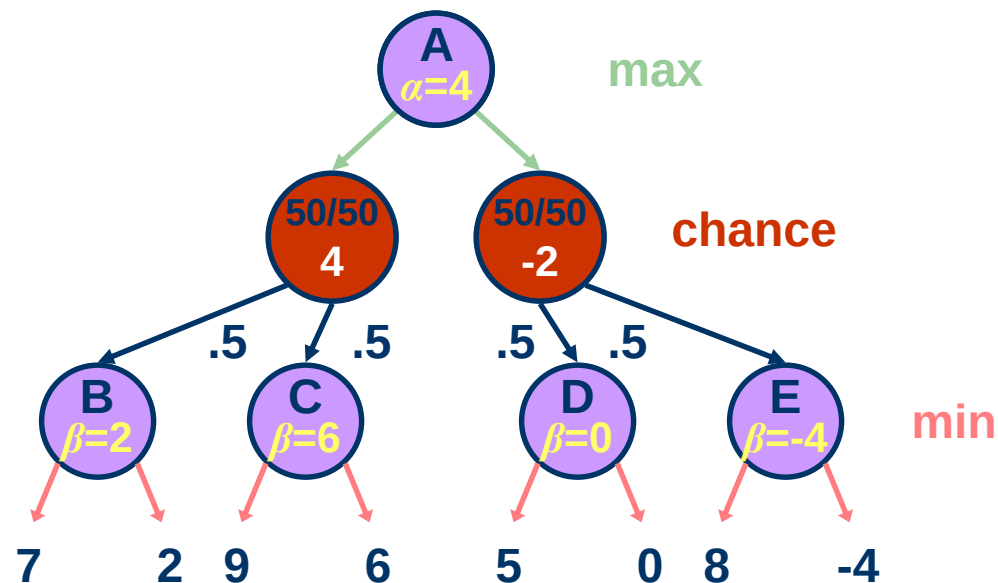
Trò chơi may rủi

- Trọng số được xác định bằng xác suất xảy ra
- Sử dụng **giá trị kỳ vọng** cho một quyết định: Tổng tất cả các nút con nhân với trọng số



Trò chơi may rủi

- Máy quyết định: Chọn nước đi với giá trị kỳ vọng lớn nhất



Trò chơi may rủi

- Tính may rủi làm tăng độ phân nhánh và độ sâu của trò chơi
- Phép thử alpha-beta kém hiệu quả

DEEPBLUE

“Deep Blue” (IBM)

- Bộ xử lý song song, 32 nút
- Mỗi nút tích hợp 8 VLSI “chess chips”
- Có thể tìm kiếm 200 million cấu hình/giây
- Sử dụng minimax, alpha-beta, heuristic nguy hiểm
- Độ sâu 14
- Chống hiệu ứng chân trời bằng cách tìm kiếm tới độ sâu 40
- Sử dụng thư viện nước đi

DEEPBLUE

Kasparov với Deep Blue, May 1997

- Chơi 6 ván, trọng tài ACM
- Kasparov thua với kết quả 2 thắng & 1 hoà và 3 thua
- Đây là sự kiện lịch sử vì lần đầu tiên máy tính trở thành kiện tướng vô địch cờ vua

Bảng xếp hạng cờ vua



Máy tính trong các trò chơi khác

- **Cờ**
 - Vô địch thế giới trong **Chinook**
 - Đánh bại bất kỳ người nào, (Tinsley ,1994)
 - Sử dụng alpha-beta, thư viện nước đi (> 443 billion)
- **Othello**
 - Máy tính dễ dàng đánh bại người chơi
- **Go**
 - Độ phân nhánh $b \sim 360$
 - 2 tỷ đô la cho người viết chương trình đánh bại chuyên gia con người